ASSIGNMENT 1

Design, Implement and Validate a City Metro Smart-Card Fare System

CO1 - C++ constructs and object-oriented programming

Student	Submission details
Name	D Hoshea Paul
Register number	192471019
Course / slot	DSA0110 / Slot B
Prepared	20 October 2026
Compiler	TDM64 GCC 4.9.2, C++14

GitHub repository

https://github.com/hosheapaul2006/DSA0110_SLOT-B-192471019-

12 completed tasks | 27 execution cases | 89 baseline/extension assertions

Includes explanations, pseudocode, UML diagrams, predictions, test tables, actual outputs, complexity comparisons and complete source listings.

Prepared with Codex assistance. Predictions and pseudocode were recorded before the corresponding execution/implementation. Review and individual verification remain necessary for the viva.

Contents

Execution and design overview 3

Task 1 - Understand the problem and identify the objects 4

Task 2 - Tokens, keywords, identifiers and constants 6

Task 3 - Data types and type compatibility 8

Task 4 - Control implicit conversions 10

Task 5 - Expressions, precedence and associativity 12

Task 6 - Design the control structures 14

Task 7 - From procedural to object-oriented 16

Task 8 - Read and print data safely 18

Task 9 - Implement and test the complete fare logic 21

Task 10 - Compare expectations with execution 25

Task 11 - Recommend and defend the final design 27

Task 12 - Re-evaluate and adapt the design 29

Complete C++ source listings 33

Sources, provenance and viva preparation 49

Execution and design overview

This report addresses all 12 tasks in the supplied 13-page Assignment 1 brief. The original model prices by farthest zone and uses integer paise, half-open peak intervals, a fixed reserve and a validated card API. Task12 modifies pricing and pass entitlement while retaining that balance core.

The executed submission contains 19 C++ source files: 18 working programs and one deliberately ill-formed narrowing diagnostic. All working programs ran, with 27 execution cases and one expected compile rejection. A final rebuild/retest passed all 28 cases. The baseline suite passes 52 assertions and the changed-requirement suite passes 37. No Classroom submission is claimed; the student chose to submit the link personally.

Student/senior concession rates were unspecified in the PDF. The agreed assumptions are 10%/20%, one category per trip, concession after the applicable fare adjustment, and one final nearest-paisa rounding with half-paisa ties up. Task12 weekend pricing replaces peak; pass validity is a calendar month and every traversed zone must be selected. The Rs50 reserve applies even to zero-fare pass travel.

The exact separate SIMATS pseudocode guide was unavailable. Conventional structured notation is used, as agreed; exact compliance with that missing guide is not claimed. The brief requested a differently named repository; the student selected the existing course repository shown on the cover. Its name is preserved, while the local deliverable uses DSA01-192471019-Assignment1.

Evidence screenshots display genuine captured program stdout/stderr in an HTML transcript. They are browser screenshots, not native terminal captures. Every task also retains machine-readable execution records and plain text output. Source hashes and pre-execution prediction commits provide additional traceability; commit dates reflect actual session work and were not backdated.

Build and reproduce

```
g++ -std=c++14 -Wall -Wextra -pedantic -O2 Task09/main.cpp -o build/Task09_main.exe
python scripts/verify.py
python scripts/verify.py --task 9
```

Create build/ before manual compilation. The installed compiler is C:\Program Files (x86)\Dev-Cpp\MinGW64\bin\g++.exe. Task8 uses the supplied input files. Task11 runs from its folder with a configuration filename argument. README.md and each runs.json give the complete reproduction instructions.

Task 1 - Understand the problem and identify the objects

Object analysis and justification

Object	Data	Behaviour and justification
SmartCard	Private balance in paise	Checks and atomically deducts an affordable fare; validates recharges. One owner of balance prevents scattered checks and accidental negative values.
Passenger	Name and category	Supplies eligibility for a regular, student or senior fare. It is separate from the card because eligibility belongs to the person.
Journey	Farthest zone, hour, minute, category	Describes one fare request. A value record keeps transaction inputs together without owning the balance.
Zone	A scoped enumeration	Identifies a permitted network region. It is a value, not a class with an unnecessary mutable fare.
FareTable	Shared constexpr fare data	Validates a zone and computes a quote without changing the card. Separating pricing from payment permits independent testing.

The UML source is `objects.mmd`; the report includes a rendered class diagram. Association: a passenger uses a card, a journey supplies a passenger category and zone, and the fare table produces the amount the card attempts to deduct.

OOP principles

Encapsulation is the private `SmartCard::balance_` and checked public methods. Abstraction is `quote()` and `deduct()`: callers request pricing/payment without manipulating percentage arithmetic or stored balances. Inheritance could express a common `FarePolicy` interface with regular and pass policy implementations. Polymorphism could select a policy through that interface. The small baseline uses enum categories and composition instead: three data-driven rates do not justify a passenger subclass hierarchy. These are reasoned possible uses, not a claim that every OOP principle must be forced into the baseline.

Rules and assumptions

Money is nonnegative integer paise. A valid journey leaves at least 5000 paise; a denied transaction leaves the balance unchanged. Only known zones and valid 24-hour times are accepted. Peak is [08:00,10:00) or [17:00,20:00), so 10 AM and 8 PM are off-peak. One category applies per journey; assumed student/senior concessions are 10%/20%. Fare calculations round only the final nonnegative fraction to the nearest paisa, ties up. Recharges must be positive and remain within the documented application balance limit of 10^12 paise. This limit also makes arithmetic bounds explicit. The problem prices by farthest zone, not distance; real station routing is outside this baseline.

Procedural alternative and complexity

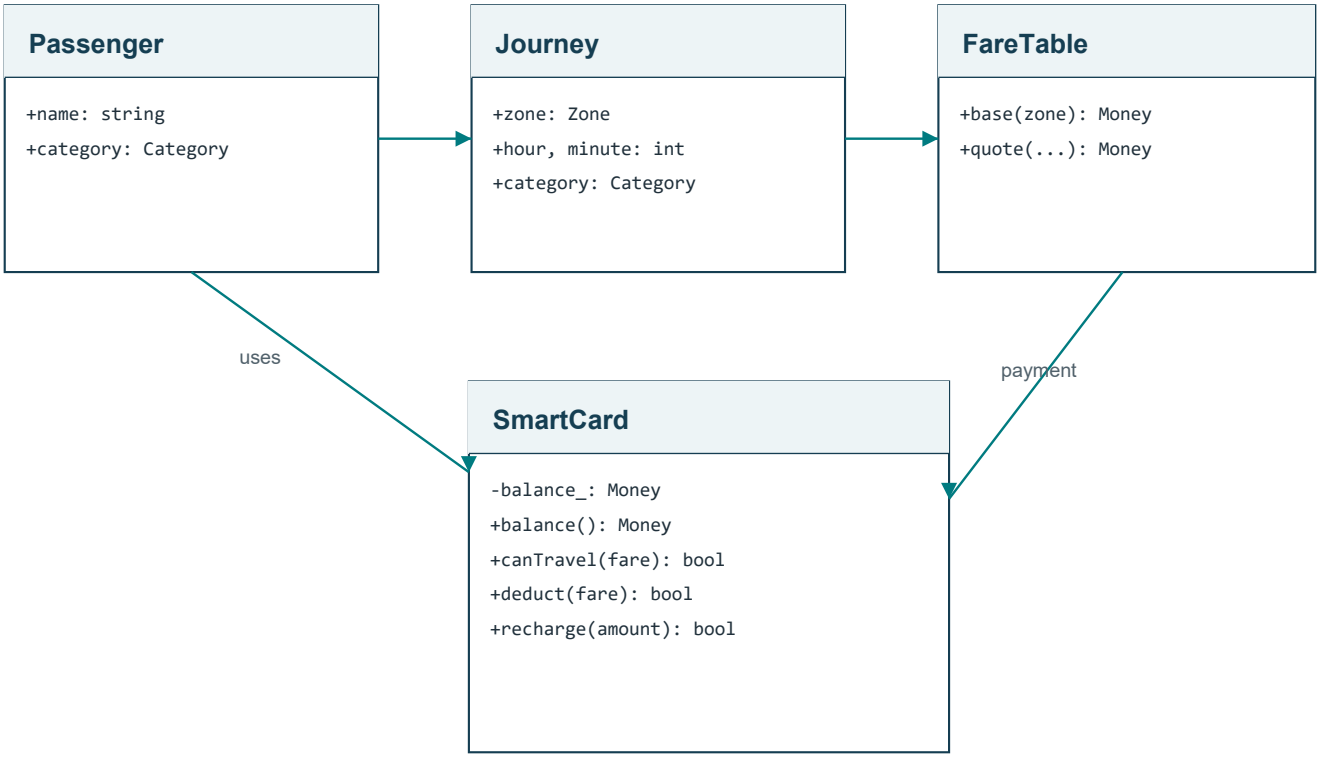
```
GLOBAL balance, fareTable, minimumBalance
FUNCTION deduct(fare)
    IF fare < 0 OR balance < minimumBalance THEN RETURN false
    IF fare > balance - minimumBalance THEN RETURN false
    balance <- balance - fare
    RETURN true
END FUNCTION
```

Procedural code can also centralise a fare constant, so OOP is not automatically faster or cheaper to maintain. With global mutable balance, any function can bypass the guard; with the class, all writes pass through `deduct()` or `recharge()`. A minimum-balance revision changes one constant in either carefully factored design. An unstructured procedural design with k copied checks needs O(k) edits, whereas this design has one policy definition. A fare quote and payment are O(1) time and O(1) extra space. Each card stores one `long long`, independent of the number of journeys; n cards occupy O(n) storage. The fare table is shared rather than copied into every card.

Verification

`main.cpp` executes the object collaboration: Rs 100 pays an Outer peak fare of Rs 36, leaving Rs 64. A subsequent Rs 30 deduction is denied and leaves Rs 64. The program checks these invariants and returns nonzero on failure. Actual output and build diagnostics are recorded beside the source.

UML class structure



Class diagram corresponds to the implemented baseline/extension responsibilities. Full Mermaid source is included in the task folder.

Recorded execution outputs

main.cpp - objects

```
Passenger: D Hoshea Paul
Fare Rs 36.00
Journey allowed: true
Balance Rs 64.00
Unaffordable trip denied: true
Balance unchanged: true
```

Actual exit status: 0

Task 2 - Tokens, keywords, identifiers and constants

Token classification

Kind requested in the assignment	Examples in this program	Meaning
Keyword	`constexpr`, `const`, `enum`, `class`, `return`, `int`	Reserved language words; cannot be variable names.
Identifier	`BASE_PAISE`, `FareTable`, `safeZone`, `quote`	Names for data, types and functions.
Constant/literal	`1000`, `20`, `0`, `' '`, string literals	Values appearing in source. A named constexpr object is an identifier token referring to a constant.
Operator	`=`, `<<`, `==`, `::`	Initialisation syntax, stream insertion/comparison and scope qualification.
Punctuator	`;`, `{`, `}`, `(`, `)`, `,`	Delimit declarations, statements, scopes and argument lists.

The exercise's five groups are pedagogical: the formal language grammar also distinguishes literals and groups many operators with punctuators. `#define` is a preprocessing directive, not a C++ keyword. Comments are not runtime tokens.

Constants and zone representation

`include/metro.hpp` is the authoritative baseline set of three fares, surcharge, concession assumptions and minimum balance. `constexpr std::array<Money,3>` gives typed entries with a known bound. A fare revision changes this one definition; the quote function reads it rather than repeating numeric fares in branches. The fixed Task6 skeleton keeps its mandated constants because the task explicitly forbids edits outside its TODOs. Demonstration/test expected values are independent oracles, not additional operational fare definitions.

A macro is textual substitution before compilation: it has no namespace or type, and arithmetic macros need parentheses. `const` prevents later assignment but can be initialised from runtime input. `constexpr` requires a constant expression and can be used by `static_assert` and compile-time computations. Prefer constexpr for this fixed fare table; prefer validated runtime data for faculty-approved configuration revisions (Task11).

An unscoped `enum` leaks enumerators into the enclosing scope and implicitly converts to an integer; `LegacyInner` demonstrates that conversion. `enum class` keeps names under `Zone::` and requires an explicit cast for indexing. It prevents accidental comparison with an unrelated enum or integer. A cast can still create an invalid enumeration value, so public quote methods validate their input.

Names use letters/digits/underscores, cannot start with a digit or equal a keyword, and are case-sensitive. Avoid reserved implementation names: double underscores and underscore-uppercase combinations, plus leading underscores at global scope. Classes use PascalCase, variables/functions camelCase, and constants UPPER_SNAKE_CASE consistently here.

Macro alternative and complexity

```
#define DEMO_PEAK_PERCENT 20
fare <- base * (100 + DEMO_PEAK_PERCENT) / 100
```

The executed demonstration compares macro, const and constexpr values. The macro creates no object/storage by itself. A constexpr scalar often needs no storage when its value is substituted; taking its address can require storage.

The fare array can occupy storage when indexed at runtime: 3 * sizeof(long long), measured as 24 bytes on this compiler. Constants are not automatically all storage-free. Typed and macro versions both have O(1) time per fare and O(1) space for this fixed table. Changing token mechanisms does not change the algorithm's complexity.

Recorded execution outputs

main.cpp - constants

```
Fares (paise): 1000 2000 3000
Macro/const/constexpr: 20 20 20
Legacy enum index: 0
Scoped zone index (explicit): 0
Minimum Rs 50.00
Student Inner peak Rs 10.80
```

Actual exit status: 0

Task 3 - Data types and type compatibility

Chosen representations

System value	Type	Reason
Fare, balance, daily collection	`long long` paise (`Money`)	Exact decimal currency at paisa resolution; daily collections exceed 32-bit paise.
Peak flag, payment result	`bool`	Only true/false states.
Display character	`char`	One character, such as a separator; not a money type.
Hour/minute	`int`	Small range, validated before use; permits detection of negative input.
Zone	`enum class Zone`	Scoped, typed identifiers; explicitly validated on external boundaries.
Journey count	`long long`	More than sufficient for 3,000,000 and future accumulation.
Container index/size	`std::size_t` (unsigned)	Matches array indexing after validation; avoid comparing it directly with negative input.
Precision demonstration	`float`, `double`	Measured sizes/ranges; floating currency is examined, not used for card balances.
Inferred count	`auto journeys = 3000000LL`	`auto` is deduction, not a new numeric type; here it is long long.

The executed program prints `sizeof`, lowest and maximum values for all requested types. Do not assume every platform has these sizes: the measured compiler gives bool/char 1 byte, int/unsigned/float 4 bytes, long long/double 8 bytes. Char's range is printed as integers so it is not confused with character glyphs. Floating `lowest()` is the most negative finite value; `min()` would mean the smallest positive normal value and would be misleading here. Actual ranges are in the captured output, including the values from `<climits>`.

Compatibility and conversions

In `10 + 0.5 + true`, the int is converted to double and the bool contributes 1, converted to double for addition. The result is double 11.5. Integral promotions turn many bool/char operands into int; the usual arithmetic conversions then choose a common arithmetic type. Implicit floating-to-integer conversion drops the fractional part and can be invalid if outside the destination range. Explicit braces and validated casts make intention visible (Task4).

Precision and overflow experiment

At the maximum baseline peak fare of 3600 paise, 3,000,000 journeys collect 10,800,000,000 paise. A measured 32-bit int has INT_MAX 2,147,483,647, so it is insufficient; long long is sufficient. The first unsafe 3600-paise addition is number 596,524. In whole rupees, 108,000,000 fits int, but mixing units would be a serious error. The program detects the mathematical limit without executing undefined signed overflow.

Ordinary Rs10/Rs20/Rs30 fares and their whole-rupee peak prices are exactly representable as binary double at these totals. Therefore the program also adds Rs36 and reports zero error: the assignment's suspected rounding problem must not be invented when inputs do not exhibit it. A separately labelled diagnostic fare of Rs10.10 (1010 paise, not an official zone fare) exposes representation error at n = 20,000; 500,000; 3,000,000. Its actual double total is compared with the exact integer-paise total using long double for the difference. Error need not grow monotonically or with a fixed sign; repeated rounding depends on the running sum. Task10 repeats the comparison

with an actual concessional fare mixture rather than only this diagnostic value.

Alternative and complexity

With double rupees the loop is `total += fareRupees`; with integer paise it is `total += farePaise`. Each addition takes O(1) time and a whole run O(n) time; each accumulator uses O(1) space. Both long long and double occupy 8 bytes here, but that is an observation, not a universal C++ guarantee. Integer overflow must still be prevented; exact representation does not mean unlimited range. For bounded fare inputs and these journey counts, long long paise avoids both the observed floating accumulation error and the 32-bit collection overflow.

Recorded execution outputs

main.cpp - sizes_and_precision

```
bool bytes=1 lowest=0 max=1
char bytes=1 lowest=-128 max=127
int bytes=4 lowest=-2147483648 max=2147483647
long long bytes=8 lowest=-9223372036854775808 max=9223372036854775807
unsigned bytes=4 lowest=0 max=4294967295
float bytes=4 lowest=-3.40282e+038 max=3.40282e+038
double bytes=8 lowest=-1.79769e+308 max=1.79769e+308
auto journeys is long long: true
INT_MAX=2147483647 LLONG_MAX=9223372036854775807
Network maximum baseline total paise=10800000000
First 3600-paise addition exceeding int: 596524
n=20000 diagnostic Rs10.10 total=202000.000000071974 exact paise=20200000 error rupees=0.000000071974
    whole-Rs36 error=0.000000000000
n=500000 diagnostic Rs10.10 total=5049999.999995739199 exact paise=505000000 error rupees=-0.000004260801
    whole-Rs36 error=0.000000000000
n=3000000 diagnostic Rs10.10 total=30300000.001558292657 exact paise=3030000000 error rupees=0.001558292657
    whole-Rs36 error=0.000000000000
int + double + bool = 11.500000000000
```

Actual exit status: 0

Task 4 - Control implicit conversions

Prediction before execution

`prediction.txt` and the original `predict.cpp` are committed before this program is executed. The runner verifies that the current committed prediction matches the file, then records the prediction commit and SHA-256 alongside the actual output. The exact prediction on this ASCII-compatible target is:

```
3 3.5
C 68
1 true
s >= u
```

Explain every line

1. `a / b` divides two ints, so truncation occurs before the result 3 converts to double for `c`. Casting `a` to double changes the division; `b` converts to double and `d` becomes 3.5. The default stream format prints 3 rather than 3.0.

2. `'A'` promotes from char to int before adding 2. The int result converts back to char for `ch`, giving C in this character set. `ch + 1` promotes to int and prints numeric 68. Numeric character codes are target-dependent; this output is justified by the measured ASCII-compatible environment, not universal C++.

3. Nonzero int 5 converts to true. The stream first prints its default numeric representation 1, then `std::boolalpha` changes subsequent bool formatting to true. It does not change the stored value.

4. The usual arithmetic conversions convert signed -1 to unsigned because unsigned int and int have equal rank. On this 32-bit target it becomes 4,294,967,295, so `s < u` is false. The given program intentionally receives the signed/unsigned comparison warning under `-Wall -Wextra`.

Other conversion points include lvalue-to-rvalue reads and the array-to-pointer conversion of string literals used by stream insertion. The baseline fare model explicitly converts validated enums to array indices; small int percent operands convert to Money in multiplication. `isPeak()` yields bool directly. The I/O reader first parses long long and range-checks before a cast to hour/zone.

Resolution and explicit alternative

`corrected.cpp` casts both division operands to double before division and checks `s < 0` before an unsigned cast, preserving the intended mathematical comparison. It uses brace initialisation and explicit char/bool conversions. Its expected changes are `3.5 3.5` on the first line and `s < u` on the last line. A mere `static_cast<unsigned>(s)` without the negative guard would explicitly preserve the original mistake rather than fix it. An explicit cast states intent; it is not a runtime range check.

`narrowing_fail.cpp` deliberately tries `int x{3.7};`. It is compiled with `-pedantic-errors` in addition to the ordinary flags and must be rejected. It has no executable and is never run. The diagnostics provide evidence that braces catch this narrowing at compile time. Replacing it with `int x{static_cast<int>(3.7)};` would explicitly request truncation to 3, but would not prove a floating input lies in range. `bool{5}` likewise needs an intentional conversion or a Boolean comparison.

Complexity and observed comparison

All versions perform a fixed number of operations: O(1) time and O(1) storage. Ordinary representation conversions usually compile to no extra work or a small fixed instruction sequence; no asymptotic change occurs. Compile-time narrowing errors prevent execution, while signedness warnings require the programmer to correct the policy. `execution.json` records whether prediction and actual output agree; any deliberate changes in the corrected program are explained above and are not retroactive edits to the prediction.

Recorded execution outputs

predict.cpp - prediction_check

```
3 3.5
C 68
1 true
s >= u
```

Actual exit status: 0. Prediction matches actual output.

Compiler warning: comparison between signed and unsigned integer expressions [-Wsign-compare]. Full diagnostics are preserved with the source.

corrected.cpp - explicit_conversions

```
3.5 3.5
C 68
1 true
s < u
```

Actual exit status: 0

narrowing_fail.cpp - expected compile rejection

```
Task04\narrowing_fail.cpp: In function 'int main()':
Task04\narrowing_fail.cpp:3:14: error: narrowing conversion of '3.7000000000000002e+0' from 'double' to
    'int' inside { } [-Wnarrowing]
     int x{3.7};
              ^
```

Compilation rejected as intended; no executable was run.

Task 5 - Expressions, precedence and associativity

Prediction before execution

The original `predict.cpp` and this exact `prediction.txt` are committed before running. The runner enforces that order and records the comparison.

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Operator-by-operator explanation

1. Multiplication binds more tightly than addition: 5 + (3 * 2) is 11. Parentheses force (5 + 3) * 2 to be 16. Insertion operators group left-to-right to print the values and separators.

2. Postfix x++ supplies the old value 5 and increments x to 6; prefix ++y increments y to 4 and supplies 4. Thus z = 9 and the next line prints 9 6 4. Precedence describes grouping, not general evaluation order. These two operands modify different variables, so their relative evaluation order does not change this result.

3. `x > y` is evaluated for the conditional operator, choosing x = 6. % and * have the same precedence and associate left-to-right: (6 % 4) * 2 = 4. The parentheses around the whole conditional avoid accidental grouping with <<.

4. Unary minus binds before / or %. Integer division truncates toward zero: 10/3 = 3 and -10/3 = -3. The corresponding remainders are 1 and -1, satisfying a = (a/b)*b + a%b. A nonzero remainder has the dividend's sign, not necessarily the divisor's. Division by zero and INT_MIN/-1 are excluded from valid inputs.

5. i takes 0, 3, 6, 9; `k += i` accumulates 18. Relational < forms the loop condition. Compound assignment groups right-to-left and updates its left operand once; it is not a comparison. The loop body and increment execute in the for-loop's defined order.

x++ is convenient when the old value is required; ++x when the new value is required. Avoid `i = i++`: in the C++14 mode used here, the modifications are unsequenced and the expression has undefined behaviour. In C++17 and later, assignment sequencing changes its status, but it still assigns the old value back and fails to express an intended increment. It is discussed, not executed.

Readable fare expressions

```
peak <- (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour < 20)
multiplier <- peak ? 120 : 100
rate <- category == Student ? 10 : (category == Senior ? 20 : 0)
fare <- ROUND_HALF_UP(base * multiplier * (100-rate), 10000)
balance -= fare      // only after the affordability guard
```

Comparisons bind more tightly than &&, which binds more tightly than ||. Thus the unparenthesised peak expression in the assignment is equivalent, but explicit interval parentheses make boundary review easier. Logical operators short-circuit. `?:` has lower precedence than those logical/comparison operators and groups right-to-left; nested conditionals require care. Multiplication and division group left-to-right, so division is postponed until the complete bounded numerator is formed; dividing an integer percentage first would lose fractional information. The peak surcharge is followed by the assumed concession, with one final rounding operation.

Lookup alternative and complexity

`expressions.cpp` builds `bool peak[24]` and compares all entries with the expression, then checks the four required boundary hours and rejects hour 24. A checked table index is essential: negative/24 indices are invalid memory accesses. Once initialised, either check is O(1) time and O(1) space for a fixed 24-hour day. The table occupies `24*sizeof(bool)`, measured as 24 bytes here; the expression requires no table. Initialisation costs O(24). A table may use fewer comparisons, but cache/compiler effects mean it is not automatically faster. Minute-based schedules would need more entries or interval logic.

Recorded execution outputs

predict.cpp - prediction_check

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Actual exit status: 0. Prediction matches actual output.

expressions.cpp - peak_lookup

```
Lookup vs expression matches: 24/24
Lookup storage bytes: 24
Hour 8 peak=true
Hour 10 peak=false
Hour 17 peak=true
Hour 20 peak=false
Student Outer peak Rs 32.40
Senior Outer offpeak Rs 24.00
Invalid hour rejected: true
```

Actual exit status: 0

Task 6 - Design the control structures

`pseudocode.txt` was committed before the Task6 C++ source was written. It states inputs, outputs, validation, zone selection, fare computation, affordability and daily iteration. It uses conventional structured notation; the separate SIMATS standard was unavailable, as agreed before implementation.

Complete the Code

`complete.cpp` retains the supplied headers, enum, constants, main and printing logic. Only the two TODO regions are completed. TODO1 uses switch, break and integer paise. Its default rejects an invalid enum with -1; TODO2 rejects negative fares and guards subtraction. For valid inputs the switch and surcharge produce 1000, 3600, 2000 paise for the given sequence. Starting at 10000 paise yields 9000 then 5400; the last fare is denied because only 3400 would remain.

The exact required output is compared by the runner, including spacing:

```
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
```

The original program has only a bool peak field and no hour field. To respect the instruction not to alter its main, the additional 10:00 and 20:00 journeys are executed in `boundaries.cpp`. Both use Outer, an initial balance of Rs80, off-peak fare Rs30, and a successful final balance of exactly Rs50. This separates the mandated unchanged example from the required additional boundary evidence.

Why each control structure is used

Use if/else for validation and mutually exclusive allow/deny actions. A switch clearly expresses the three named zone cases and prevents accidental fallthrough with break (or a direct return). With no matching case and no default, control continues after the switch; a function must still return a valid result. Here the invalid-zone branch prevents an uninitialised or arbitrary fare. The conditional operator is suitable for the small peak multiplier selection; avoid nested side-effecting conditionals for payment.

A range-based for is simplest for a known batch of trips, because it avoids indexing mistakes and directly supplies each trip. An indexed for is useful when a row number is required for reports/tests. A while loop is suitable for streaming input until EOF, not for endlessly retrying EOF. `continue` skips a validated-invalid input without changing balance; `break` can end a stream on a user stop command or exit a switch. Neither should silently skip a payment failure that must be recorded. One day with n journeys uses O(n) time and O(1) processing space when streamed; storing all journeys first uses O(n) memory.

Array alternative and complexity

`FareTable::base()` checks the index then directly reads the array; the boundary program compares all three array results with a switch using the same constants. For fixed zones both are O(1) per lookup. A compiler may emit comparisons or a jump table for switch, so there is no universal speed winner. A three-entry long long array occupies 24 bytes on this compiler. With z configurable zones, the table requires O(z) storage but lookup remains O(1) after range validation. Adding Airport requires an enum entry plus a table entry; switch implementations also need a new case at every switch location. Task12 avoids those repeated price branches by modifying the design toward one checked table.

Pseudocode recorded before C++

```
ALGORITHM ComputeAndPayFare(zone, hour, minute, category, balance)
INPUT: zone, 24-hour time, passenger category, balance in paise
OUTPUT: fare in paise, allowed/denied status, new balance
BEGIN
    IF zone is unknown OR hour NOT IN [0,23] OR minute NOT IN [0,59] THEN
        OUTPUT "Invalid journey"
        RETURN
    END IF
    CASE zone OF
        Inner:  base <- BASE_PAISE[0]
        Middle: base <- BASE_PAISE[1]
        Outer:  base <- BASE_PAISE[2]
    END CASE
    peak <- (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour < 20)
    multiplier <- 100
    IF peak THEN multiplier <- 120 END IF
    concession <- 0
    IF category = Student THEN concession <- 10 END IF
    IF category = Senior THEN concession <- 20 END IF
    numerator <- base * multiplier * (100 - concession)
    fare <- ROUND_HALF_UP(numerator / 10000)
    IF balance >= MIN_BALANCE_PAISE AND fare <= balance - MIN_BALANCE_PAISE THEN
        balance <- balance - fare
        OUTPUT fare, "Allowed", balance
    ELSE
        OUTPUT fare, "Denied", balance
    END IF
END

ALGORITHM ProcessDay(journeys)
INPUT: a finite sequence of journeys
OUTPUT: one result per valid journey and an invalid-input message otherwise
BEGIN
    FOR EACH journey IN journeys DO
        IF journey is invalid THEN
            OUTPUT "Skipped invalid journey"
            CONTINUE
        END IF
        CALL ComputeAndPayFare(journey inputs)
    END FOR
END

Notation: conventional structured pseudocode. The separate SIMATS Engineering
style guide was not provided or found, so exact institutional conformance is
not asserted. This pseudocode is saved in Git before writing the Task6 C++.
The mandated skeleton has regular passengers only; the general algorithm above
also states the agreed concession extension used by the shared fare system.
```

Recorded execution outputs

complete.cpp - given_example

```
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
```

Actual exit status: 0

boundaries.cpp - extra_journeys

```
10:00 peak=false fare Rs 30.00 allowed=true balance Rs 50.00
20:00 peak=false fare Rs 30.00 allowed=true balance Rs 50.00
Switch/array and time checks: 5/5
```

Actual exit status: 0

Task 7 - From procedural to object-oriented

Two working versions

`procedural.cpp` keeps a separate global balance and free `canTravel()` and `deduct()` functions. `object_oriented.cpp` uses the shared SmartCard, whose balance is private. Both execute the same four requests: 1000, 3600, 2000 and -1 paise from an initial 10000. The expected/actual sequence is accepted, accepted, denied, denied, ending at 5400 paise. The runner checks exact matching output for the two versions, including the measured 8-byte balance storage.

The initial procedural test caught an output-order bug: a single C++14 stream expression both called deduct and read the balance. Evaluation order allowed the old balance to be formatted first. Payment now occurs in its own statement before printing, and the same correction is applied to the OO version. The final exact output checks verify the corrected reports. This illustrates why side effects and observations should be separated even when the numeric payment logic is sound.

How the class protects the balance

`canTravel()` first rejects a negative fare and a balance below the reserve, then compares fare with `balance - minimum`. This avoids overflowing an expression such as `minimum + fare` or subtracting an arbitrary huge fare. `deduct()` calls the guard itself, so callers cannot forget to check first. On denial it changes nothing. `recharge()` rejects nonpositive values and tests the upper bound using `amount <= maximum - balance` before addition. Constructor validation prevents negative or over-limit initial balances. A card may start below Rs50 and be recharged, but cannot travel until the reserve is satisfied.

Private storage prevents ordinary external code from directly assigning a negative balance; access control is a compile-time boundary. This is not a claim of protection against memory corruption or concurrent unsynchronised writes. The gate prototype is single-threaded. A real shared account service would need atomic transactions/locking and persistent audit records.

Which design changes more easily?

Both well-factored versions use the same MIN_BALANCE_PAISE constant. A reserve revision therefore takes one policy edit in either version, and both need the same tests rerun. The OO advantage is that clients cannot legally bypass the policy by changing the field directly. The global procedural version exposes balance to all code in its translation unit and couples callers to that shared state. A procedural version with opaque state and a carefully controlled API could obtain many of the same protections; OOP is not a substitute for good factoring. Multiple card objects naturally hold independent balances, whereas the single-global example would need additional parameters or containers.

Public-member alternative

`public_balance.cpp` deliberately uses `struct PublicCard { long long balance; }` and assigns -1. It compiles and runs: the language allows the write even though the application rule is broken. The output records both the broken value and the public/private object sizes. Exposing a balance would allow any caller to evade the Rs50 reserve, manufacture credit or subtract a negative fare.

Both private and public representations store one long long and have no virtual methods, so they have the same measured storage here (8 bytes). The standard does not promise every independently designed class layout will be identical. Both guarded deduct functions take O(1) time and O(1) extra space; access control does not add a per-operation algorithmic cost. Inlining can eliminate wrapper call overhead. A public field is therefore not a justified performance trade-off for abandoning the central balance invariant.

Recorded execution outputs

procedural.cpp - payments

```
Fare paise 1000 accepted=true balance Rs 90.00
Fare paise 3600 accepted=true balance Rs 54.00
Fare paise 2000 accepted=false balance Rs 54.00
Fare paise -1 accepted=false balance Rs 54.00
Balance storage bytes: 8
```

Actual exit status: 0

object_oriented.cpp - payments

```
Fare paise 1000 accepted=true balance Rs 90.00
Fare paise 3600 accepted=true balance Rs 54.00
Fare paise 2000 accepted=false balance Rs 54.00
Fare paise -1 accepted=false balance Rs 54.00
Balance storage bytes: 8
```

Actual exit status: 0

public_balance.cpp - invariant_violation

```
Public member allows balance=-1 paise
Public/private object bytes: 8/8
```

Actual exit status: 0

Task 8 - Read and print data safely

Safe input design

`main.cpp` uses the shared `readInteger()` routine with `std::cin`. It performs formatted extraction and checks `in.fail()` explicitly. On letters or an unrepresentable integer, it clears the error state and uses `ignore(numeric_limits<streamsize>::max(), '\n')` to discard the bad line before retrying. Clearing without discarding would repeatedly encounter the same bad token. EOF terminates with exit status 2, rather than retrying forever. Values are range-checked as long long before the explicit cast to a zone/hour/category.

After a successful numeric prefix, the routine consumes the rest of the line and rejects non-whitespace characters. Consequently `10junk` is not silently accepted as 10. This is stricter than plain `cin >> hour`. Input convention is one value per line, with balance in integer paise; arbitrary decimal currency input is not implicitly rounded. A real user-facing kiosk could add a validated decimal-text parser without changing the exact-money model.

Output formatting

The journey report uses `left` and `setw` to align status/fare/balance columns. `metro::money()` prints the whole-rupee part and a two-digit remainder using `setw(2)` and `setfill('0')`. Each call has its own stream, so fill state does not leak into the report. No double is used to compute or format the actual fare. `fixed` plus `setprecision(2)` is demonstrated separately on a harmless display value: precision changes floating output formatting, not representation or arithmetic accuracy. `setw` applies to the next field; fill/precision flags persist.

Test cases and expected results

Fixture	Deliberate condition	Expected verified result
input.txt	Letters for balance/hour; zone 9; trailing junk; minute 60	Recover, accept Middle/Senior at 10:00; fare Rs16.00; balance Rs84.00.
exact_reserve.txt	Rs60, Inner at 10:00, regular	Allowed, Rs10.00 fare, exactly Rs50.00 left.
low_balance.txt	Rs59.99, same journey	Denied; Rs59.99 unchanged.
eof.txt	Immediate EOF	Clear termination, exit 2.
line_input.txt	stoi errors, huge integer, bad suffix, then valid 10	Four rejection messages followed by Accepted hour 10.

The runner checks expected fragments/exact outputs and exit statuses, rather than regarding successful compilation as proof of recovery. Actual outputs and input fixtures are retained for reproduction.

getline/stoi alternative and complexity

`getline_stoi.cpp` reads each complete line, calls `stoi` inside try/catch, checks the consumed-character index and then validates the hour. Without that index check, `stoi("10junk")` would also accept 10. `invalid_argument` catches absent numbers; `out_of_range` catches values outside int and invalid hour limits. A bad line is already consumed, so no cin fail-state reset is needed for a conversion exception; EOF still must be handled. For long long money use `stoll` rather than `stoi` so parsing matches the chosen representation.

For a line of length L, scanning and discarding/parsing cost O(L) in either approach. A basic direct extractor with discard can use O(1) extra program storage, while getline stores O(L). This implementation's strict suffix check also stores the remainder in a string, so its worst-case buffer space is O(L). The fixed number of fare fields makes computation O(1) after parsing; input cost depends on the characters consumed, including rejected lines.

Recorded execution outputs

main.cpp - invalid_recovery

Input fixture: input.txt

```
abc
10000
9
1
twenty
10junk
10
60
0
2

Balance in paise: Invalid number; try again
Balance in paise: Zone (0 Inner, 1 Middle, 2 Outer): Out of range; try again
Zone (0 Inner, 1 Middle, 2 Outer): Hour (0-23): Invalid number; try again
Hour (0-23): Unexpected trailing characters; try again
Hour (0-23): Minute (0-59): Out of range; try again
Minute (0-59): Category (0 Regular, 1 Student, 2 Senior):
JOURNEY REPORT
Status          Fare (Rs)       Balance (Rs)
Allowed         16.00           84.00
Display precision example: 1.00
```

Actual exit status: 0

main.cpp - exact_reserve

Input fixture: exact_reserve.txt

```
6000
0
10
0
0

Balance in paise: Zone (0 Inner, 1 Middle, 2 Outer): Hour (0-23): Minute (0-59): Category (0 Regular, 1
    Student, 2 Senior):
JOURNEY REPORT
Status          Fare (Rs)       Balance (Rs)
Allowed         10.00           50.00
Display precision example: 1.00
```

Actual exit status: 0

main.cpp - low_balance

Input fixture: low_balance.txt

```
5999
0
10
0
0

Balance in paise: Zone (0 Inner, 1 Middle, 2 Outer): Hour (0-23): Minute (0-59): Category (0 Regular, 1
    Student, 2 Senior):
JOURNEY REPORT
Status          Fare (Rs)       Balance (Rs)
Denied          10.00           59.99
Display precision example: 1.00
```

Actual exit status: 0

main.cpp - eof

```
Balance in paise: End of input
```

Actual exit status: 2

getline_stoi.cpp - strict_lines

Input fixture: line_input.txt

```
abc
24
999999999999999999999999
10junk
10

Invalid integer line
Integer/hour out of range
Integer/hour out of range
Invalid integer line
Accepted hour 10
```

Actual exit status: 0

Task 9 - Implement and test the complete fare logic

`include/metro.hpp` is the working baseline implementation. `main.cpp` is an executable verification driver with independently specified expected values. Every row prints CASE, EXPECTED, ACTUAL and PASS/FAIL; any failure gives a nonzero exit status. `test_results.csv` is generated from that actual output, not from assumed success. Tests use fresh cards for independent cases, then one shared card for the five-journey sequence.

Required normal and boundary test expectations

Case	Fare in paise	Initial balance	Expected outcome/balance
Inner/Middle/Outer off-peak	1000 / 2000 / 3000	10000 each	Allow: 9000 / 8000 / 7000
Inner/Middle/Outer peak	1200 / 2400 / 3600	10000 each	Allow: 8800 / 7600 / 6400
Inner at 08:00 and 17:00	1200	10000	Allow: 8800
Inner at 10:00 and 20:00	1000	10000	Allow: 9000
Inner at 07:59 / 09:59 / 16:59 / 19:59	1000 / 1200 / 1000 / 1200	10000 each	Allow: 9000 / 8800 / 9000 / 8800
Exact Rs50 after Inner off-peak	1000	6000	Allow: 5000
One paisa less available	1000	5999	Deny: 5999 unchanged
Student Inner peak	1080	10000	Allow: 8920
Senior Outer peak	2880	10000	Allow: 7120

Invalid inputs and invariant checks

Unknown zones -1, 3 and 99, hours -1/24, minute 60 and category 99 must throw a validation exception before pricing/payment. Negative initial balances must be rejected. Negative/zero recharges and recharges beyond the application cap must fail without mutation. A positive one-paisa recharge must succeed. LLONG_MAX recharge/fare requests must be rejected without overflow. Negative fares must not manufacture credit. A stream with letters then 8 must recover to a valid hour, demonstrating the same input routine used by Task8.

Rounding helper tests exercise half-paisa ties up and values just below a tie, even though the currently configured zone/discount combinations yield whole paise. This separates testing a future pricing rule from inventing fractional paise in the official fare table.

Five journeys calculated before execution

Starting balance = 20000 paise (Rs200). The manually calculated fares are:

1. Inner at 11:00 regular: 1000.

2. Middle at 08:00 regular: 2000 * 120/100 = 2400.

3. Outer at 10:00 regular: 3000.

4. Inner at 17:00 student: 1000 * 120/100 * 90/100 = 1080.

5. Outer at 19:00 senior: 3000 * 120/100 * 80/100 = 2880.

Total = 1000 + 2400 + 3000 + 1080 + 2880 = **10360 paise (Rs103.60)**. Final balance = **9640 paise (Rs96.40)**. Each fare and payment is compared with these independent expectations, in addition to checking the final totals.

Table-driven alternative and complexity

The ordinary fare cases already demonstrate the alternative: a table of input records and expected fares/outcomes is processed in a range-based for loop. A new boundary case adds one row rather than copying assertion code. Irregular exception/state tests use separate checks because their setup differs.

For t tests with constant-size records, either repeated handwritten checks or a table loop costs O(t) time. The stored table occupies O(t) space; a streamed test file can reduce working storage to O(1), excluding output. Independent tests still need explicit oracles: a loop comparing the implementation to itself would not establish correctness. Each individual fare/payment remains O(1).

Complete expected-versus-actual results

CASE	EXPECTED	ACTUAL	RESULT
Inner offpeak	10.00,allow,90.00	10.00,allow,90.00	PASS
Middle offpeak	20.00,allow,80.00	20.00,allow,80.00	PASS
Outer offpeak	30.00,allow,70.00	30.00,allow,70.00	PASS
Inner peak	12.00,allow,88.00	12.00,allow,88.00	PASS
Middle peak	24.00,allow,76.00	24.00,allow,76.00	PASS
Outer peak	36.00,allow,64.00	36.00,allow,64.00	PASS
07:59	10.00,allow,90.00	10.00,allow,90.00	PASS
08:00	12.00,allow,88.00	12.00,allow,88.00	PASS
09:59	12.00,allow,88.00	12.00,allow,88.00	PASS
10:00	10.00,allow,90.00	10.00,allow,90.00	PASS
16:59	10.00,allow,90.00	10.00,allow,90.00	PASS
17:00	12.00,allow,88.00	12.00,allow,88.00	PASS
19:59	12.00,allow,88.00	12.00,allow,88.00	PASS
20:00	10.00,allow,90.00	10.00,allow,90.00	PASS
Exactly Rs50 remains	10.00,allow,50.00	10.00,allow,50.00	PASS
One paisa below reserve	10.00,deny,59.99	10.00,deny,59.99	PASS
Student Inner peak	10.80,allow,89.20	10.80,allow,89.20	PASS
Student Middle offpeak	18.00,allow,82.00	18.00,allow,82.00	PASS
Senior Outer peak	28.80,allow,71.20	28.80,allow,71.20	PASS
Senior Outer offpeak	24.00,allow,76.00	24.00,allow,76.00	PASS
Reject zone -1	1	1	PASS
Reject zone 3	1	1	PASS
Reject zone 99	1	1	PASS
Reject hour -1	1	1	PASS
Reject hour 24	1	1	PASS
Reject minute 60	1	1	PASS
Reject category 99	1	1	PASS

CASE	EXPECTED	ACTUAL	RESULT
Reject negative initial balance	1	1	PASS
Reject negative recharge	0	0	PASS
Reject zero recharge	0	0	PASS
Rejected recharges leave balance	10000	10000	PASS
Accept positive recharge	1	1	PASS
Positive recharge new balance	10001	10001	PASS
Reject recharge above application cap	0	0	PASS
Reject huge recharge	0	0	PASS
Reject negative fare	0	0	PASS
Reject huge fare safely	0	0	PASS
Letters then valid hour recovery	1	1	PASS
Half-paisa rounds up	101	101	PASS
Below half-paisa rounds down	100	100	PASS
Five-journey fare 1	1000	1000	PASS
Five-journey payment 1	1	1	PASS
Five-journey fare 2	2400	2400	PASS
Five-journey payment 2	1	1	PASS
Five-journey fare 3	3000	3000	PASS
Five-journey payment 3	1	1	PASS
Five-journey fare 4	1080	1080	PASS
Five-journey payment 4	1	1	PASS
Five-journey fare 5	2880	2880	PASS
Five-journey payment 5	1	1	PASS
Five-journey hand total paise	10360	10360	PASS
Five-journey remaining paise	9640	9640	PASS

Recorded execution outputs

main.cpp - all_tests

Actual exit status: 0

Task 10 - Compare expectations with execution

Before execution

`expectations.txt` records the repeated five-fare cycle and mathematical totals before execution. The cycle is Inner peak Rs12, Middle off-peak Rs20, Outer peak Rs36, Student Middle off-peak Rs18, Senior Outer peak Rs28.80. It totals 11480 paise. All three required counts are divisible by five, so the expected exact totals are 45,920,000; 1,148,000,000; 6,888,000,000 paise respectively. These are actual configured concessional fares, unlike the separate Task3 diagnostic.

The program checks these totals and the first would-overflow journey. Actual measurements, including differences and timings, are in `measurements.csv` and the captured output. A generated `observations.md` lists the measured numbers beside the independent expectations. Time/error values are not invented or rounded to hide differences.

Why int, long long and double differ

On this system int is 4 bytes, long long and double are 8 bytes each. The 32-bit int paise total fits 20,000 and 500,000 journeys, but fails at the network count. The cycle reaches 2,147,483,240 paise after 935,315 journeys. The next 1200-paise fare exceeds INT_MAX, so a checked int accumulator stops at journey 935,316. It never deliberately executes signed integer overflow: C++ signed overflow is undefined behaviour, and an apparently wrapped result would not be reliable evidence of a defined language result. The representability check demonstrates the failure without relying on that behaviour.

Long long paise remains exact at 6,888,000,000 and leaves ample headroom. It is still finite: overflow checks or bounded totals are necessary for arbitrary lifetime data. Double rupees has a wide numeric range but represents 28.8 only approximately; repeated additions round at changing magnitudes. Its difference from the long-double conversion of the exact paise total is measured at all three scales. This is different from Task3's Rs36 experiment, which correctly reports no double error for exact whole-rupee values at these counts.

Compiler diagnostics and actual differences

All operational examples are built with `-std=c++14 -Wall -Wextra -pedantic -O2`. The required unmodified Task4 prediction example emits `-Wsign-compare`: int -1 is compared with unsigned 1 after an unsigned conversion. Its explicit guarded comparison is warning-free. The deliberate brace-narrowing example is rejected with `-Wnarrowing` under the additional `-pedantic-errors` flag. It is an expected compile-time error, not an unexecuted required working program.

The final operational sources compile without warnings. Warnings alone did not catch the Task7 reporting-order issue: a runtime exact-output test detected it, and separating deduction from printing resolved it. Task4 and Task5 predictions match actual output exactly; their deliberate corrected/alternative versions are explained separately. Numeric accumulation is the measured expectation-versus- execution difference in this task.

Complexity and measurements

Every scalar addition is O(1) time and each accumulator is O(1) space. Processing n fares is O(n) time. Long long/double do not change that complexity; their constants and numerical semantics differ. The benchmark times the integer and double loops separately with steady_clock and volatile accumulators so the compiler cannot replace the whole accumulation with a constant. Modulo indexing, volatile reads/writes and loop overhead are included. It is one local run, not a controlled repeated statistical benchmark or an end-to-end gate throughput test. The totals and type limits justify the correctness recommendation; tiny timing differences do not justify abandoning exact money.

Measured expectations versus execution

These values come from the recorded execution on this machine.

Journeys	Expected/actual integer paise	Actual double rupees	Difference in rupees	Checked int fits
20000	45920000	459199.999999969092	-0.000000030908	yes
500000	1148000000	11480000.000010456890	0.000010456890	yes
3000000	6888000000	68879999.999236658216	-0.000763341784	no

The network double error is about -0.0763 paisa in this run. The exact paise totals match the hand calculations. The integer and floating loop timings are in measurements.csv and include volatile access and loop overhead.

Recorded execution outputs

main.cpp - measured_totals

```
Accumulator bytes: int=4 long long=8 double=8
Cycle fares paise: 1200 2000 3600 1800 2880
journeys,exact_paise,double_rupees,error_rupees,int_us,double_us,int_safe,first_int_overflow
20000,45920000,459199.999999969092,-0.000000030908,12,51,yes,0
500000,1148000000,11480000.000010456890,0.000010456890,292,1293,yes,0
3000000,6888000000,68879999.999236658216,-0.000763341784,1768,7738,no,935316
Microbenchmark includes volatile reads/writes and loop overhead; not a gate throughput guarantee.
Hand total / overflow checks: PASS
```

Actual exit status: 0

Task 11 - Recommend and defend the final design

Recommendation based on experiments

Retain integer-paise Money (`long long`), scoped zone/category enums, one typed fare table, named percentage/reserve constants, a pure quote operation and a SmartCard with a private validated balance. Compose these responsibilities; do not create inheritance hierarchies for every zone or passenger category. Keep quote separate from payment so denied trips do not accidentally change state. Validate inputs before casts, check arithmetic bounds and round the final percentage result once. Use table-driven tests for ordinary cases and explicit state/exception tests for invalid data.

Evidence: Task9 passes 52 independent checks, including all zone/time cases and the hand total Rs103.60. Task7's private/public examples have the same measured 8-byte object size, while only the public representation permits direct negative assignment. Task10 exactly collects 6,888,000,000 paise at 3,000,000 mixed journeys; int paise fails at journey 935,316. The measured double total differs slightly, so a compact floating representation offers no correctness benefit. Task8 demonstrates rejection and recovery rather than relying on valid input alone.

Carrying out a fare revision

For the small fixed baseline, edit the one BASE_PAISE definition, rebuild, rerun normal/boundary/concession tests, and deploy the tested binary. Add a fare-policy version to a production deployment and audit the active configuration. Test expected values must be revised independently from the policy so that tests do not merely repeat a changed implementation. The mandated standalone Task6 skeleton remains a teaching snapshot, not a second deployed pricing service.

Runtime configuration alternative

`config.cpp` implements a separate validated table loaded once at startup. A configuration gives z followed by z positive paise amounts (1 <= z <= 32), with a conservative per-fare limit preventing arithmetic overflow. An unreadable file, bad count, missing/negative fare or trailing data fails closed. The table is fully validated before use; a partial load is never installed as a working tariff.

The normal fixture gives Outer peak Rs36.00; a separate demonstration revision to 1100/2200/3300 paise gives Rs39.60 without recompiling. Five invalid fixture/ file cases are executed and must exit 2. This is a concrete prototype of the alternative, not a silent revision of the official assignment fares. It changes base fares only; concessions, passes and reserve are still application rules.

Reading z bounded numeric values costs O(z) startup time, or O(B) when accounting for B file characters, and O(z) storage. Once loaded, an indexed fare quote is O(1). Compiled constexpr data avoids file parsing and load failure but requires rebuild/redeployment after a revision. Runtime configuration reduces that cost but adds schema, validation, version consistency and tamper-protection concerns. For this assignment retain compiled constants; for a network with frequent fare changes, adopt a centrally versioned, validated configuration service.

Network correctness and limitations

The type/range evidence establishes numerical capacity for the specified daily load. Per-journey work is O(1), so n journeys take O(n) processing time with O(1) streaming working storage; c active cards require O(c) state. The loop timings do not establish actual gate latency or distributed throughput. This prototype does not persist accounts, authenticate cards, prevent simultaneous gate races, implement secure configuration distribution, or resolve real station routes. A production network needs transactional balance updates, durable audit logs, fault handling and fare-policy rollout coordination. Its correctness cannot be claimed solely from a fast local arithmetic loop.

Concession/pass conventions are disclosed assumptions requiring policy approval, not facts asserted by the PDF. The following Task12 modifies pricing and entitlements while retaining the tested money and card invariants.

Recorded execution outputs

Case / file	Configuration input	Actual captured result	Exit
baseline_config / fares.txt	3 / 1000 2000 3000	Configured zones: 3; Outer peak fare Rs 36.00; Allowed=true balance Rs 64.00	0
revised_config / revised_fares.txt	3 / 1100 2200 3300	Configured zones: 3; Outer peak fare Rs 39.60; Allowed=true balance Rs 60.40	0
negative_config / negative_fare.txt	3 / 1000 -2000 3000	Configuration rejected: Invalid/missing fare	2
missing_fare_config / missing_fare.txt	3 / 1000 2000	Configuration rejected: Invalid/missing fare	2
trailing_config / trailing_data.txt	3 / 1000 2000 3000 unknown	Configuration rejected: Unexpected trailing configuration data	2
invalid_count / invalid_count.txt	999	Configuration rejected: Invalid zone count or unreadable file	2
missing_file / does_not_exist.txt	(file absent)	Configuration rejected: Invalid zone count or unreadable file	2

All seven rows ran config.cpp from Task11 with the listed filename argument. Normal/revised configuration outputs were compared exactly; rejected cases were checked for their expected message and exit status.

Task 12 - Re-evaluate and adapt the design

A. Impact analysis (2 marks)

The new Airport zone requires a fourth enum value and a 6000-paise fare. The baseline's three-case switch and three-entry bounds cannot price it without modification. The fare adjustment expression must choose weekend OR peak, because 10% weekend reduction and the 20% peak surcharge cannot combine. Monthly passes need selected-zone data and a calendar-month validity check that a simple passenger concession cannot represent. A single farthest-zone field is not enough to decide pass coverage; journeys now carry origin and destination zones.

`include/metro_extended.hpp` adds a four-entry enum/table, Date, a route Journey, MonthlyPass and FareRules. It reuses baseline Money, roundHalfUp, concession rates, peak checks and SmartCard. The original three fares are referenced from the baseline constexpr array rather than retyped. Task6's required skeleton and the baseline are preserved as the evaluated original version; the extension is explicitly separate rather than silently rewriting previous evidence.

B. Evaluation of the original design (2 marks)

The original switch could absorb Airport with a fourth case, but every copy of that selection would need auditing. It cannot alone express pass entitlement or weekend priority without mixing several concerns. Adding a public pass flag to SmartCard would also be insufficient: a flag cannot encode two zones, expiry or route coverage. Conversely, private balance and checked deduct already accept any nonnegative fare, including zero, so they do not need redesign.

Worked comparisons: Airport weekday at 08:00 is 6000 * 120/100 = 7200 paise; Airport Saturday at 08:00 is 6000 * 90/100 = 5400, not 6480 from incorrectly combining weekend and peak. Student Airport Saturday is 6000 * 90/100 * 90/100 =

4860. A valid Inner/Middle pass yields zero for that fully covered route, while Middle/Outer must pay its normal fare. A pass selecting Inner/Outer does not cover the intervening Middle zone on an Inner-to-Outer route. These examples identify exactly which assumptions the earlier single-zone design could not represent.

C. Design decision and implementation (2 marks)

MODIFY the design: retain the exact-money/card core, use a checked table for four-zone pricing, and separate monthly entitlement from fare adjustments.

```
INPUT journey, monthlyPass, card
VALIDATE route zones, actual calendar date, time, passenger category
IF pass month/year match AND every traversed zone is selected THEN
    fare <- 0
ELSE
    base <- fareTable[MAX(originZone, destinationZone)]
    IF calendar date is Saturday or Sunday THEN adjustment <- 90
    ELSE IF peak time THEN adjustment <- 120
    ELSE adjustment <- 100
    END IF
    fare <- ROUND_HALF_UP(base * adjustment * (100-concession) / 10000)
END IF
CALL card.deduct(fare)  // retains Rs50 minimum, even for a pass
```

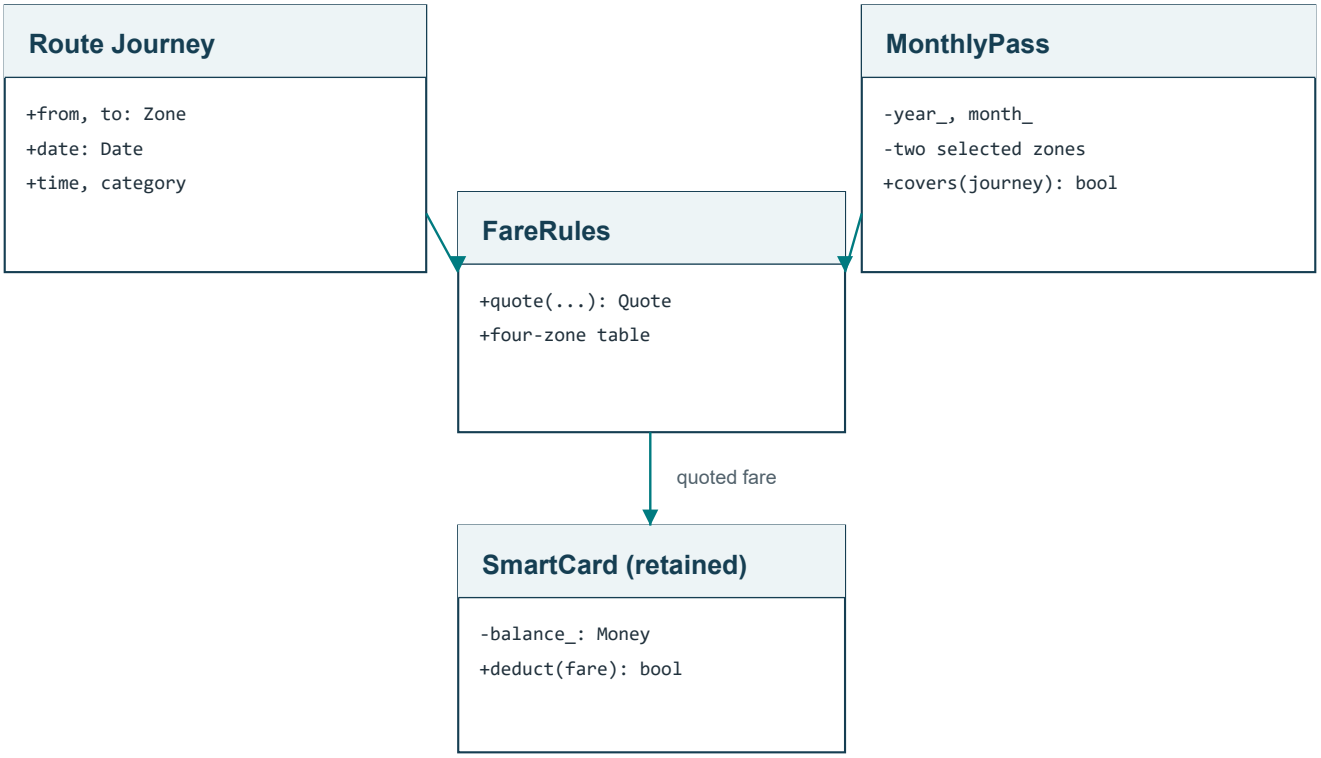
The route model assumes zones form the ordered corridor Inner-Middle-Outer- Airport; all intermediate zones are traversed. This is documented, not claimed as a full city railway graph. A pass has exactly two distinct selected zones, valid from the first through last day of its stated calendar month/year. Outside coverage or validity it falls back to ordinary pricing. Weekend status is derived from a validated Gregorian date, so an arbitrary caller Boolean cannot misclassify a weekday. A covered pass takes priority over discounts/surcharges; the Rs50 reserve remains mandatory. No pass purchase price was supplied, so purchase/payment for the pass is outside the prototype.

D. Evidence-based defence (2 marks)

`main.cpp` prints expected and actual values for Airport regular/peak/weekend, student and senior prices, pass coverage, both month boundaries, expiry, wrong year, uncovered routes, reverse routes, same-zone travel, and retained baseline fares. It also tests exact/insufficient reserve, an unchanged denied balance, 1000 consecutive zero-fare pass journeys, leap-year validity and invalid route/ time/date/pass inputs. The full executed comparison table is `test_results.csv`. Task9's original suite is retained so new evidence does not erase baseline tests.

The separate policy makes revision easier without weakening payment. Table lookup, calendar checking and two-zone membership are O(1) for the fixed four zones; a route coverage loop visits at most four entries. For a general ordered network with z zones and k traversed zones, pricing lookup remains O(1), the table occupies O(z), and coverage checking is O(k); membership in the two-zone pass is O(1). A real graph would add route discovery cost. Per-card/pass state remains O(1), and n journeys still take O(n) for this fixed model. Additional date/zone fields cost fixed storage; no lifetime trip history is stored for an unlimited pass. The extension is more complex than the original switch, but the tests show that complexity corresponds to required new policy rather than a wholesale rewrite of proven balance handling.

UML class structure



Class diagram corresponds to the implemented baseline/extension responsibilities. Full Mermaid source is included in the task folder.

Complete expected-versus-actual results

CASE	EXPECTED	ACTUAL	RESULT
Airport weekday offpeak	60.00,pass=0,weekend=0,peak=0	60.00,pass=0,weekend=0,peak=0	PASS
Airport weekday peak	72.00,pass=0,weekend=0,peak=1	72.00,pass=0,weekend=0,peak=1	PASS
Airport Saturday peak replaced	54.00,pass=0,weekend=1,peak=0	54.00,pass=0,weekend=1,peak=0	PASS
Airport Sunday offpeak	54.00,pass=0,weekend=1,peak=0	54.00,pass=0,weekend=1,peak=0	PASS
Student Airport weekend	48.60,pass=0,weekend=1,peak=0	48.60,pass=0,weekend=1,peak=0	PASS
Senior Airport peak	57.60,pass=0,weekend=0,peak=1	57.60,pass=0,weekend=0,peak=1	PASS
Pass covered weekday	0.00,pass=1,weekend=0,peak=0	0.00,pass=1,weekend=0,peak=0	PASS
Pass covered weekend	0.00,pass=1,weekend=0,peak=0	0.00,pass=1,weekend=0,peak=0	PASS
Pass last day	0.00,pass=1,weekend=0,peak=0	0.00,pass=1,weekend=0,peak=0	PASS
Pass first day	0.00,pass=1,weekend=0,peak=0	0.00,pass=1,weekend=0,peak=0	PASS
Pass next month expires	18.00,pass=0,weekend=1,peak=0	18.00,pass=0,weekend=1,peak=0	PASS
Pass wrong year	24.00,pass=0,weekend=0,peak=1	24.00,pass=0,weekend=0,peak=1	PASS
Pass uncovered third zone	36.00,pass=0,weekend=0,peak=1	36.00,pass=0,weekend=0,peak=1	PASS
Nonadjacent pass crosses Middle	30.00,pass=0,weekend=0,peak=0	30.00,pass=0,weekend=0,peak=0	PASS
Same selected zone covered	0.00,pass=1,weekend=0,peak=0	0.00,pass=1,weekend=0,peak=0	PASS
Baseline Inner retained	10.00,pass=0,weekend=0,peak=0	10.00,pass=0,weekend=0,peak=0	PASS
Baseline Middle retained	20.00,pass=0,weekend=0,peak=0	20.00,pass=0,weekend=0,peak=0	PASS
Baseline Outer retained	30.00,pass=0,weekend=0,peak=0	30.00,pass=0,weekend=0,peak=0	PASS
Pass at reserve accepted	1	1	PASS
Pass reserve unchanged	5000	5000	PASS
Pass below reserve denied	0	0	PASS
Airport peak exact reserve	1	1	PASS
Airport balance exact reserve	5000	5000	PASS
Airport one paisa short denied	0	0	PASS
Airport denied balance unchanged	12199	12199	PASS
1000 pass trips have no count limit	1	1	PASS
2026-10-24 is weekend	1	1	PASS

CASE	EXPECTED	ACTUAL	RESULT
2026-10-20 is weekday	0	0	PASS
Leap day 2024 valid	1	1	PASS
Nonleap day 2026 invalid	0	0	PASS
Century 1900 not leap	0	0	PASS
Century 2000 leap	1	1	PASS
Reject invalid date even with pass	1	1	PASS
Reject unknown fourth-zone enum	1	1	PASS
Reject invalid time even with pass	1	1	PASS
Reject invalid pass month	1	1	PASS
Reject duplicate pass zones	1	1	PASS

Recorded execution outputs

main.cpp - extension_tests

Actual exit status: 0

Complete C++ source listings

These are the saved, compiled sources. Lines are wrapped only for page layout; use the repository files for compilation. The original Task4/Task5 prediction programs and Task6 skeleton originate in the assignment. Other implementation and explanations were prepared with Codex assistance.

include/checks.hpp

```
#ifndef CHECKS_HPP
#define CHECKS_HPP
#include <iostream>
#include <sstream>
#include <string>
class Checks {
    int total_=0, failed_=0;
public:
    Checks() { std::cout << "CASE|EXPECTED|ACTUAL|RESULT\n"; }
    template<class T> void equal(const std::string& label, const T& expected, const T& actual) {
        ++total_;
        const bool pass=expected==actual;
        if (!pass) ++failed_;
        std::cout << label << '|' << expected << '|' << actual << '|' << (pass?"PASS":"FAIL") << '\n';
    }
    int finish() const {
        std::cout << "SUMMARY " << total_-failed_ << '/' << total_ << " passed\n";
        return failed_==0 ? 0 : 1;
    }
};
#endif
```

include/metro.hpp

```
#ifndef METRO_HPP
#define METRO_HPP
#include <array>
#include <iomanip>
#include <limits>
#include <sstream>
#include <stdexcept>
#include <string>

namespace metro {
using Money = long long;
enum class Zone { Inner, Middle, Outer };
enum class Category { Regular, Student, Senior };
constexpr std::array<Money, 3> BASE_PAISE{{1000, 2000, 3000}};
constexpr int PEAK_PERCENT = 20;
constexpr int STUDENT_PERCENT = 10; // Assumption: the assignment does not set rates.
constexpr int SENIOR_PERCENT = 20;
constexpr Money MIN_BALANCE_PAISE = 5000;
constexpr Money MAX_BALANCE_PAISE = 1000000000000LL; // Explicit application limit.

inline bool validZone(Zone z) {
    const int index = static_cast<int>(z);
    return index >= 0 && index < static_cast<int>(BASE_PAISE.size());
}
inline bool isPeak(int hour, int minute = 0) {
    if (hour < 0 || hour > 23 || minute < 0 || minute > 59)
        throw std::invalid_argument("Invalid time");
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}
inline int concession(Category category) {
    switch (category) {
        case Category::Regular: return 0;
        case Category::Student: return STUDENT_PERCENT;
        case Category::Senior: return SENIOR_PERCENT;
    }
    throw std::invalid_argument("Unknown passenger category");
}
inline Money roundHalfUp(Money numerator, Money denominator) {
    if (numerator < 0 || denominator <= 0)
        throw std::invalid_argument("Invalid nonnegative ratio");
    const Money quotient = numerator / denominator;
    const Money remainder = numerator % denominator;
    return quotient + (remainder >= denominator / 2 + denominator % 2 ? 1 : 0);
}
class FareTable {
public:
    Money base(Zone zone) const {
        if (!validZone(zone)) throw std::invalid_argument("Unknown zone");
        return BASE_PAISE[static_cast<std::size_t>(zone)];
    }
    Money quote(Zone zone, int hour, Category category = Category::Regular, int minute = 0) const {
        const int multiplier = isPeak(hour, minute) ? 100 + PEAK_PERCENT : 100;
        // Multiply bounded fares and round once, after all percentages.
        return roundHalfUp(base(zone) * multiplier * (100 - concession(category)), 10000);
    }
};
struct Passenger { std::string name; Category category; };
struct Journey { Zone zone; int hour; int minute; Category category; };
class SmartCard {
    Money balance_;
public:
    explicit SmartCard(Money initial) : balance_(initial) {
        if (initial < 0 || initial > MAX_BALANCE_PAISE)
            throw std::invalid_argument("Invalid initial balance");
    }
    Money balance() const { return balance_; }
    bool canTravel(Money fare) const {
        return fare >= 0 && balance_ >= MIN_BALANCE_PAISE && fare <= balance_ - MIN_BALANCE_PAISE;
    }
    bool deduct(Money fare) {
        if (!canTravel(fare)) return false;
        balance_ -= fare;
        return true;
    }
    bool recharge(Money amount) {
```

```
        if (amount <= 0 || amount > MAX_BALANCE_PAISE - balance_) return false;
        balance_ += amount;
        return true;
    }
};
inline std::string money(Money amount) {
    if (amount < 0) throw std::invalid_argument("Cannot print negative money");
    std::ostringstream out;
    out << amount / 100 << '.' << std::setw(2) << std::setfill('0') << amount % 100;
    return out.str();
}
inline bool readInteger(std::istream& in, std::ostream& out, const std::string& prompt,
                        long long minimum, long long maximum, long long& value) {
    for (;;) {
        out << prompt;
        in >> value;
        if (in.fail()) {
            if (in.eof()) { out << "End of input\n"; return false; }
            out << "Invalid number; try again\n";
            in.clear();
            in.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            continue;
        }
        std::string rest;
        std::getline(in, rest);
        if (rest.find_first_not_of(" \t\r") != std::string::npos) {
            out << "Unexpected trailing characters; try again\n";
            continue;
        }
        if (value < minimum || value > maximum) {
            out << "Out of range; try again\n";
            continue;
        }
        return true;
    }
}
}
#endif
```

include/metro_extended.hpp

```
#ifndef METRO_EXTENDED_HPP
#define METRO_EXTENDED_HPP
#include "metro.hpp"
#include <algorithm>
#include <array>
namespace extended {
using metro::Money;
enum class Zone { Inner, Middle, Outer, Airport };
constexpr Money AIRPORT_PAISE=6000;
constexpr int WEEKEND_PERCENT=10;
constexpr std::array<Money,4>
    FARES{{metro::BASE_PAISE[0],metro::BASE_PAISE[1],metro::BASE_PAISE[2],AIRPORT_PAISE}};
struct Date { int year,month,day; };
inline bool validDate(Date d) {
    if (d.year<1 || d.year>9999 || d.month<1 || d.month>12) return false;
    const int days[]={31,28,31,30,31,30,31,31,30,31,30,31};
    const bool leap=d.year%4==0 && (d.year%100!=0 || d.year%400==0);
    const int limit=days[d.month-1]+(d.month==2 && leap ? 1:0);
    return d.day>=1 && d.day<=limit;
}
inline bool isWeekend(Date d) {
    if (!validDate(d)) throw std::invalid_argument("Invalid date");
    // Gregorian calendar weekday calculation: 0=Sunday, 6=Saturday.
    const int offsets[]={0,3,2,5,0,3,5,1,4,6,2,4};
    int y=d.year;
    if (d.month<3) --y;
    const int weekday=(y+y/4-y/100+y/400+offsets[d.month-1]+d.day)%7;
    return weekday==0 || weekday==6;
}
inline bool validZone(Zone z) { const int value=static_cast<int>(z); return value>=0 && value<4; }
struct Journey { Zone from,to; Date date; int hour,minute; metro::Category category; };
inline void validate(const Journey& j) {
    if (!validZone(j.from) || !validZone(j.to)) throw std::invalid_argument("Unknown route zone");
    if (!validDate(j.date)) throw std::invalid_argument("Invalid date");
    (void)metro::isPeak(j.hour,j.minute);
    (void)metro::concession(j.category);
}
class MonthlyPass {
    bool active_=false;
    int year_=0,month_=0;
    Zone first_=Zone::Inner,second_=Zone::Middle;
public:
    MonthlyPass() = default;
    MonthlyPass(int year,int month,Zone first,Zone second)
        : active_(true),year_(year),month_(month),first_(first),second_(second) {
        if (!validDate(Date{year,month,1}) || !validZone(first) || !validZone(second) || first==second)
            throw std::invalid_argument("Pass requires a valid month and two distinct zones");
    }
    bool covers(const Journey& j) const {
        validate(j);
        if (!active_ || year_!=j.date.year || month_!=j.date.month) return false;
        const int start=std::min(static_cast<int>(j.from),static_cast<int>(j.to));
        const int finish=std::max(static_cast<int>(j.from),static_cast<int>(j.to));
        for (int z=start;z<=finish;++z)
            if (z!=static_cast<int>(first_) && z!=static_cast<int>(second_)) return false;
        return true;
    }
};
struct Quote { Money fare; bool passApplied,weekendApplied,peakApplied; };
class FareRules {
public:
    Quote quote(const Journey& j,const MonthlyPass& pass) const {
        validate(j);
        if (pass.covers(j)) return Quote{0,true,false,false};
        const bool weekend=isWeekend(j.date);
        const bool peak=!weekend && metro::isPeak(j.hour,j.minute);
        const int adjustment=weekend ? 100-WEEKEND_PERCENT : (peak ? 100+metro::PEAK_PERCENT : 100);
        const auto
    index=static_cast<std::size_t>(std::max(static_cast<int>(j.from),static_cast<int>(j.to)));
        const Money
    fare=metro::roundHalfUp(FARES[index]*adjustment*(100-metro::concession(j.category)),10000);
        return Quote{fare,false,weekend,peak};
    }
};
}
```

```
#endif
```

Task01/main.cpp

```
#include "../include/metro.hpp"
#include <iostream>
int main() {
    const metro::Passenger passenger{"D Hoshea Paul", metro::Category::Regular};
    const metro::Journey journey{metro::Zone::Outer, 8, 0, passenger.category};
    const metro::FareTable table;
    metro::SmartCard card{10000};
    const auto fare = table.quote(journey.zone, journey.hour, journey.category, journey.minute);
    std::cout << "Passenger: " << passenger.name << '\n';
    std::cout << "Fare Rs " << metro::money(fare) << '\n';
    std::cout << "Journey allowed: " << std::boolalpha << card.deduct(fare) << '\n';
    std::cout << "Balance Rs " << metro::money(card.balance()) << '\n';
    const auto before = card.balance();
    const bool denied = !card.deduct(3000);
    std::cout << "Unaffordable trip denied: " << denied << '\n';
    std::cout << "Balance unchanged: " << (before == card.balance()) << '\n';
    return denied && before == card.balance() && card.balance() >= 5000 ? 0 : 1;
}
```

Task02/main.cpp

```
#include "../include/metro.hpp"
#include <iostream>
#define DEMO_PEAK_PERCENT 20
enum LegacyZone { LegacyInner, LegacyMiddle, LegacyOuter };
int main() {
    const int runtimeConstant = metro::PEAK_PERCENT;
    constexpr int compileTimeConstant = metro::PEAK_PERCENT;
    static_assert(compileTimeConstant == 20, "Peak rate must match assignment");
    const int legacyIndex = LegacyInner; // Unscoped enum converts implicitly.
    const auto safeZone = metro::Zone::Inner;
    const metro::FareTable table;
    std::cout << "Fares (paise):";
    for (const auto value : metro::BASE_PAISE) std::cout << ' ' << value;
    std::cout << "\nMacro/const/constexpr: " << DEMO_PEAK_PERCENT << ' '
              << runtimeConstant << ' ' << compileTimeConstant << '\n';
    std::cout << "Legacy enum index: " << legacyIndex << '\n';
    std::cout << "Scoped zone index (explicit): " << static_cast<int>(safeZone) << '\n';
    std::cout << "Minimum Rs " << metro::money(metro::MIN_BALANCE_PAISE) << '\n';
    std::cout << "Student Inner peak Rs " << metro::money(table.quote(safeZone,8,metro::Category::Student))
    << '\n';
    return table.quote(safeZone,8,metro::Category::Student) == 1080 ? 0 : 1;
}
```

Task03/main.cpp

```
#include "../include/metro.hpp"
#include <climits>
#include <iomanip>
#include <iostream>
#include <limits>
#include <type_traits>
template <class T> void sizeRow(const char* name) {
    std::cout << name << " bytes=" << sizeof(T) << " lowest="
              << std::numeric_limits<T>::lowest() << " max=" << std::numeric_limits<T>::max() << '\n';
}
int main() {
    sizeRow<bool>("bool");
    std::cout << "char bytes=" << sizeof(char) << " lowest=" << CHAR_MIN << " max=" << CHAR_MAX << '\n';
    sizeRow<int>("int"); sizeRow<long long>("long long"); sizeRow<unsigned>("unsigned");
    sizeRow<float>("float"); sizeRow<double>("double");
    auto journeys = 3000000LL;
    std::cout << "auto journeys is long long: " << std::boolalpha
              << std::is_same<decltype(journeys),long long>::value << '\n';
    std::cout << "INT_MAX=" << INT_MAX << " LLONG_MAX=" << LLONG_MAX << '\n';
    std::cout << "Network maximum baseline total paise=" << journeys * 3600LL << '\n';
    std::cout << "First 3600-paise addition exceeding int: " << INT_MAX / 3600 + 1 << '\n';
    const long long counts[] = {20000,500000,3000000};
    std::cout << std::fixed << std::setprecision(12);
    for (const auto n : counts) {
        double actual = 0.0, whole = 0.0;
        long long exact = 0;
        for (long long i=0;i<n;++i) { actual += 10.10; exact += 1010; whole += 36.0; }
        const long double reference = static_cast<long double>(exact) / 100.0L;
        std::cout << "n=" << n << " diagnostic Rs10.10 total=" << actual
                  << " exact paise=" << exact << " error rupees="
                  << static_cast<long double>(actual)-reference
                  << " whole-Rs36 error=" << whole-static_cast<double>(n*36) << '\n';
    }
    const auto mixed = 10 + 0.5 + true;
    std::cout << "int + double + bool = " << mixed << '\n';
    return journeys * 3600LL > INT_MAX && mixed == 11.5 ? 0 : 1;
}
```

Task04/corrected.cpp

```
#include <iostream>
int main() {
    const int a{7}, b{2};
    const double c{static_cast<double>(a) / static_cast<double>(b)};
    const double d{static_cast<double>(a) / static_cast<double>(b)};
    std::cout << c << ' ' << d << '\n';
    const char ch{static_cast<char>(static_cast<int>('A') + 2)};
    std::cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';
    const bool flag{static_cast<bool>(5)};
    std::cout << static_cast<int>(flag) << ' ' << std::boolalpha << flag << '\n';
    const unsigned u{1U}; const int s{-1};
    const bool less{s < 0 || static_cast<unsigned>(s) < u};
    std::cout << (less ? "s < u" : "s >= u") << '\n';
    return c == 3.5 && less ? 0 : 1;
}
```

Task04/narrowing_fail.cpp

```
// Deliberately ill-formed diagnostic example; never executed.
int main() {
    int x{3.7};
    return x;
}
```

Task04/predict.cpp

```
#include <iostream>

int main()
{
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';

    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';

    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Task05/expressions.cpp

```
#include "../include/metro.hpp"
#include <iostream>
#include <stdexcept>
int main() {
    bool peak[24] = {};
    for (int hour=0;hour<24;++hour) peak[hour] = (hour>=8 && hour<10) || (hour>=17 && hour<20);
    int matches=0;
    for (int hour=0;hour<24;++hour) if (peak[hour] == metro::isPeak(hour)) ++matches;
    std::cout << "Lookup vs expression matches: " << matches << "/24\n";
    std::cout << "Lookup storage bytes: " << sizeof(peak) << '\n';
    for (const int hour : {8,10,17,20}) std::cout << "Hour " << hour << " peak=" << std::boolalpha <<
    metro::isPeak(hour) << '\n';
    const metro::FareTable table;
    std::cout << "Student Outer peak Rs " <<
    metro::money(table.quote(metro::Zone::Outer,8,metro::Category::Student)) << '\n';
    std::cout << "Senior Outer offpeak Rs " <<
    metro::money(table.quote(metro::Zone::Outer,10,metro::Category::Senior)) << '\n';
    bool invalidRejected=false;
    try { (void)metro::isPeak(24); } catch (const std::invalid_argument&) { invalidRejected=true; }
    std::cout << "Invalid hour rejected: " << invalidRejected << '\n';
    return matches==24 && invalidRejected ? 0 : 1;
}
```

Task05/predict.cpp

```
#include <iostream>

int main()
{
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';

    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

Task06/boundaries.cpp

```
#include "../include/metro.hpp"
#include <iostream>
metro::Money switchBase(metro::Zone zone) {
    switch (zone) {
        case metro::Zone::Inner: return metro::BASE_PAISE[0];
        case metro::Zone::Middle: return metro::BASE_PAISE[1];
        case metro::Zone::Outer: return metro::BASE_PAISE[2];
    }
    throw std::invalid_argument("Unknown zone");
}
int main() {
    const metro::FareTable table;
    int passed=0;
    for (const int hour : {10,20}) {
        const auto f=table.quote(metro::Zone::Outer,hour);
        metro::SmartCard card{8000};
        const bool allowed=card.deduct(f);
        std::cout << hour << ":00 peak=" << std::boolalpha << metro::isPeak(hour)
                  << " fare Rs " << metro::money(f) << " allowed=" << allowed
                  << " balance Rs " << metro::money(card.balance()) << '\n';
        if (f==3000 && allowed && card.balance()==5000) ++passed;
    }
    for (int i=0;i<3;++i) {
        const auto z=static_cast<metro::Zone>(i);
        if (switchBase(z)==table.base(z)) ++passed;
    }
    std::cout << "Switch/array and time checks: " << passed << "/5\n";
    return passed==5 ? 0 : 1;
}
```

Task06/complete.cpp

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;     // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
    // TODO 1 completed: switch selects the fare; an invalid zone uses a sentinel.
    long long base = 0;
    switch (z) {
        case Zone::Inner: base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer: base = BASE_PAISE[2]; break;
        default: return -1;
    }
    return peak ? base + base * PEAK_SURCHARGE_PERCENT / 100 : base;
}

bool canTravel(long long balance, long long f)
{
    // TODO 2 completed: avoid subtraction until operands make it safe.
    return f >= 0 && balance >= MIN_BALANCE_PAISE && f <= balance - MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000;                      // Rs 100.00 in paise
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << "  balance Rs " << balance / 100 << '\n';
    }
}
```

Task07/object_oriented.cpp

```
#include "../include/metro.hpp"
#include <iostream>
int main() {
    metro::SmartCard card{10000};
    for (const long long fare : {1000LL,3600LL,2000LL,-1LL}) {
        const bool accepted=card.deduct(fare);
        std::cout << "Fare paise " << fare << " accepted=" << std::boolalpha << accepted
                  << " balance Rs " << metro::money(card.balance()) << '\n';
    }
    std::cout << "Balance storage bytes: " << sizeof(card) << '\n';
    const auto before=card.balance();
    const bool rechargeRejected=!card.recharge(-100);
    return before==5400 && rechargeRejected && card.balance()==before ? 0 : 1;
}
```

Task07/procedural.cpp

```
#include "../include/metro.hpp"
#include <iostream>
long long balance=10000;
bool canTravel(long long fare) {
    return fare>=0 && balance>=metro::MIN_BALANCE_PAISE && fare<=balance-metro::MIN_BALANCE_PAISE;
}
bool deduct(long long fare) {
    if (!canTravel(fare)) return false;
    balance-=fare; return true;
}
int main() {
    for (const long long fare : {1000LL,3600LL,2000LL,-1LL}) {
        const bool accepted=deduct(fare);
        std::cout << "Fare paise " << fare << " accepted=" << std::boolalpha << accepted
                  << " balance Rs " << metro::money(balance) << '\n';
    }
    std::cout << "Balance storage bytes: " << sizeof(balance) << '\n';
    return balance==5400 ? 0 : 1;
}
```

Task07/public_balance.cpp

```
#include "../include/metro.hpp"
#include <iostream>
struct PublicCard { long long balance; };
int main() {
    PublicCard card{10000};
    card.balance=-1; // Legal access, broken application invariant.
    std::cout << "Public member allows balance=" << card.balance << " paise\n";
    std::cout << "Public/private object bytes: " << sizeof(card) << '/' << sizeof(metro::SmartCard) << '\n';
    return card.balance<metro::MIN_BALANCE_PAISE ? 0 : 1;
}
```

Task08/getline_stoi.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
int main() {
    std::string line;
    while (std::getline(std::cin,line)) {
        try {
            std::size_t used=0;
            const int hour=std::stoi(line,&used);
            if (line.find_first_not_of(" \t\r",used)!=std::string::npos)
                throw std::invalid_argument("Trailing characters");
            if (hour<0 || hour>23) throw std::out_of_range("Hour range");
            std::cout << "Accepted hour " << hour << '\n';
            return 0;
        } catch (const std::invalid_argument&) {
            std::cout << "Invalid integer line\n";
        } catch (const std::out_of_range&) {
            std::cout << "Integer/hour out of range\n";
        }
    }
    std::cout << "End of input\n";
    return 2;
}
```

Task08/main.cpp

```
#include "../include/metro.hpp"
#include <iostream>
int main() {
    using metro::readInteger;
    long long balance=0, zone=0, hour=0, minute=0, category=0;
    if (!readInteger(std::cin,std::cout,"Balance in paise: ",0,metro::MAX_BALANCE_PAISE,balance) ||
        !readInteger(std::cin,std::cout,"Zone (0 Inner, 1 Middle, 2 Outer): ",0,2,zone) ||
        !readInteger(std::cin,std::cout,"Hour (0-23): ",0,23,hour) ||
        !readInteger(std::cin,std::cout,"Minute (0-59): ",0,59,minute) ||
        !readInteger(std::cin,std::cout,"Category (0 Regular, 1 Student, 2 Senior): ",0,2,category)) return
    2;
    const metro::FareTable table;
    const auto fare=table.quote(static_cast<metro::Zone>(zone),static_cast<int>(hour),
                                static_cast<metro::Category>(category),static_cast<int>(minute));
    metro::SmartCard card{balance};
    const bool allowed=card.deduct(fare);
    std::cout << "\nJOURNEY REPORT\n" << std::left << std::setw(16) << "Status"
              << std::setw(16) << "Fare (Rs)" << "Balance (Rs)\n";
    std::cout << std::setw(16) << (allowed ? "Allowed" : "Denied") << std::setw(16)
              << metro::money(fare) << metro::money(card.balance()) << '\n';
    std::cout << "Display precision example: " << std::fixed << std::setprecision(2) << 1.0 << '\n';
    return 0;
}
```

Task09/main.cpp

```
#include "../include/metro.hpp"
#include "../include/checks.hpp"
#include <climits>
#include <iostream>
#include <sstream>
#include <string>
std::string outcome(long long fare,bool allowed,long long balance) {
    return metro::money(fare)+","+(allowed?"allow":"deny")+","+metro::money(balance);
}
int main() {
    using metro::Category; using metro::Zone;
    Checks check;
    const metro::FareTable table;
    struct Case { const char* label; Zone zone; int hour,minute; Category category; long long balance,fare;
    bool allowed; long long after; };
    const Case cases[]={
        {"Inner offpeak",Zone::Inner,11,0,Category::Regular,10000,1000,true,9000},
        {"Middle offpeak",Zone::Middle,11,0,Category::Regular,10000,2000,true,8000},
        {"Outer offpeak",Zone::Outer,11,0,Category::Regular,10000,3000,true,7000},
        {"Inner peak",Zone::Inner,9,0,Category::Regular,10000,1200,true,8800},
        {"Middle peak",Zone::Middle,9,0,Category::Regular,10000,2400,true,7600},
        {"Outer peak",Zone::Outer,9,0,Category::Regular,10000,3600,true,6400},
        {"07:59",Zone::Inner,7,59,Category::Regular,10000,1000,true,9000},
        {"08:00",Zone::Inner,8,0,Category::Regular,10000,1200,true,8800},
        {"09:59",Zone::Inner,9,59,Category::Regular,10000,1200,true,8800},
        {"10:00",Zone::Inner,10,0,Category::Regular,10000,1000,true,9000},
        {"16:59",Zone::Inner,16,59,Category::Regular,10000,1000,true,9000},
        {"17:00",Zone::Inner,17,0,Category::Regular,10000,1200,true,8800},
        {"19:59",Zone::Inner,19,59,Category::Regular,10000,1200,true,8800},
        {"20:00",Zone::Inner,20,0,Category::Regular,10000,1000,true,9000},
        {"Exactly Rs50 remains",Zone::Inner,10,0,Category::Regular,6000,1000,true,5000},
        {"One paisa below reserve",Zone::Inner,10,0,Category::Regular,5999,1000,false,5999},
        {"Student Inner peak",Zone::Inner,8,0,Category::Student,10000,1080,true,8920},
        {"Student Middle offpeak",Zone::Middle,10,0,Category::Student,10000,1800,true,8200},
        {"Senior Outer peak",Zone::Outer,17,0,Category::Senior,10000,2880,true,7120},
        {"Senior Outer offpeak",Zone::Outer,20,0,Category::Senior,10000,2400,true,7600}
    };
    for (const auto& c:cases) {
        metro::SmartCard card{c.balance};
        const auto f=table.quote(c.zone,c.hour,c.category,c.minute);
        const bool allowed=card.deduct(f);
        check.equal(c.label,outcome(c.fare,c.allowed,c.after),outcome(f,allowed,card.balance()));
    }
    for (const int invalidZone : {-1,3,99}) {
        bool rejected=false;
        try { (void)table.quote(static_cast<Zone>(invalidZone),10); } catch(const std::invalid_argument&) {
    rejected=true; }
        check.equal("Reject zone "+std::to_string(invalidZone),true,rejected);
    }
    for (const int hour : {-1,24}) {
        bool rejected=false;
        try { (void)table.quote(Zone::Inner,hour); } catch(const std::invalid_argument&) { rejected=true; }
        check.equal("Reject hour "+std::to_string(hour),true,rejected);
    }
    bool badMinute=false,badCategory=false,badInitial=false;
    try { (void)table.quote(Zone::Inner,8,Category::Regular,60); } catch(const std::invalid_argument&) {
    badMinute=true; }
    try { (void)table.quote(Zone::Inner,8,static_cast<Category>(99)); } catch(const std::invalid_argument&)
    { badCategory=true; }
    try { metro::SmartCard invalid{-1}; } catch(const std::invalid_argument&) { badInitial=true; }
    check.equal("Reject minute 60",true,badMinute);
    check.equal("Reject category 99",true,badCategory);
    check.equal("Reject negative initial balance",true,badInitial);
    metro::SmartCard card{10000};
    check.equal("Reject negative recharge",false,card.recharge(-1));
    check.equal("Reject zero recharge",false,card.recharge(0));
    check.equal("Rejected recharges leave balance",10000LL,card.balance());
    check.equal("Accept positive recharge",true,card.recharge(1));
    check.equal("Positive recharge new balance",10001LL,card.balance());
    metro::SmartCard full{metro::MAX_BALANCE_PAISE};
    check.equal("Reject recharge above application cap",false,full.recharge(1));
    check.equal("Reject huge recharge",false,card.recharge(LLONG_MAX));
    check.equal("Reject negative fare",false,card.deduct(-1));
    check.equal("Reject huge fare safely",false,card.deduct(LLONG_MAX));
    std::istringstream letters{"letters\n8\n"}; std::ostringstream messages; long long hour=0;
```

```
    const bool recovered=metro::readInteger(letters,messages,"hour: ",0,23,hour);
    check.equal("Letters then valid hour recovery",true,recovered && hour==8 && messages.str().find("Invalid
     number")!=std::string::npos);
    check.equal("Half-paisa rounds up",101LL,metro::roundHalfUp(10050,100));
    check.equal("Below half-paisa rounds down",100LL,metro::roundHalfUp(10049,100));
    const metro::Journey journeys[]={
        {Zone::Inner,11,0,Category::Regular},{Zone::Middle,8,0,Category::Regular},
        {Zone::Outer,10,0,Category::Regular},{Zone::Inner,17,0,Category::Student},
        {Zone::Outer,19,0,Category::Senior}
    };
    const long long expected[]={1000,2400,3000,1080,2880};
    metro::SmartCard dayCard{20000}; long long total=0; int row=0;
    for (const auto& j:journeys) {
        const auto f=table.quote(j.zone,j.hour,j.category,j.minute);
        check.equal("Five-journey fare "+std::to_string(row+1),expected[row],f);
        check.equal("Five-journey payment "+std::to_string(row+1),true,dayCard.deduct(f));
        total+=f; ++row;
    }
    check.equal("Five-journey hand total paise",10360LL,total);
    check.equal("Five-journey remaining paise",9640LL,dayCard.balance());
    return check.finish();
}
```

Task10/main.cpp

```
#include "../include/metro.hpp"
#include <chrono>
#include <climits>
#include <iomanip>
#include <iostream>
int main() {
    const metro::FareTable table;
    const long long fares[]={table.quote(metro::Zone::Inner,8),table.quote(metro::Zone::Middle,10),
        table.quote(metro::Zone::Outer,17),table.quote(metro::Zone::Middle,10,metro::Category::Student),
        table.quote(metro::Zone::Outer,17,metro::Category::Senior)};
    const long long counts[]={20000,500000,3000000};
    bool passed=true;
    std::cout << "Accumulator bytes: int=" << sizeof(int) << " long long=" << sizeof(long long) << "
    double=" << sizeof(double) << '\n';
    std::cout << "Cycle fares paise:";
    for (const auto f:fares) std::cout << ' ' << f;
    std::cout <<
    "\njourneys,exact_paise,double_rupees,error_rupees,int_us,double_us,int_safe,first_int_overflow\n";
    for (const auto n:counts) {
        volatile long long exact=0;
        volatile double actual=0.0;
        auto start=std::chrono::steady_clock::now();
        for (long long i=0;i<n;++i) exact+=fares[i%5];
        auto middle=std::chrono::steady_clock::now();
        for (long long i=0;i<n;++i) actual+=static_cast<double>(fares[i%5])/100.0;
        auto end=std::chrono::steady_clock::now();
        const auto integerUs=std::chrono::duration_cast<std::chrono::microseconds>(middle-start).count();
        const auto doubleUs=std::chrono::duration_cast<std::chrono::microseconds>(end-middle).count();
        int checkedTotal=0; long long firstOverflow=0;
        for (long long i=0;i<n;++i) {
            const int f=static_cast<int>(fares[i%5]);
            if (checkedTotal>INT_MAX-f) { firstOverflow=i+1; break; }
            checkedTotal+=f;
        }
        const long double difference=static_cast<long double>(actual)-static_cast<long
    double>(exact)/100.0L;
        std::cout << n << ',' << exact << ',' << std::fixed << std::setprecision(12) << actual << ',' <<
    difference
                  << ',' << integerUs << ',' << doubleUs << ',' << (firstOverflow==0 ? "yes":"no") << ',' <<
     firstOverflow << '\n';
        if (exact!=(n/5)*11480 || (n==3000000 && firstOverflow!=935316)) passed=false;
    }
    std::cout << "Microbenchmark includes volatile reads/writes and loop overhead; not a gate throughput
    guarantee.\n";
    std::cout << "Hand total / overflow checks: " << (passed ? "PASS":"FAIL") << '\n';
    return passed ? 0 : 1;
}
```

Task11/config.cpp

```
#include "../include/metro.hpp"
#include <fstream>
#include <iostream>
#include <vector>
class ConfigFareTable {
    std::vector<metro::Money> fares_;
public:
    explicit ConfigFareTable(const char* path) {
        std::ifstream in{path};
        int count=0;
        if (!(in>>count) || count<1 || count>32) throw std::invalid_argument("Invalid zone count or
    unreadable file");
        fares_.reserve(static_cast<std::size_t>(count));
        for (int i=0;i<count;++i) {
            metro::Money fare=0;
            if (!(in>>fare) || fare<=0 || fare>1000000) throw std::invalid_argument("Invalid/missing fare");
            fares_.push_back(fare);
        }
        std::string extra;
        if (in>>extra) throw std::invalid_argument("Unexpected trailing configuration data");
    }
    std::size_t size() const { return fares_.size(); }
    metro::Money quote(std::size_t index,int hour) const {
        if (index>=fares_.size()) throw std::invalid_argument("Unknown configured zone");
        return metro::roundHalfUp(fares_[index]*(metro::isPeak(hour)?120:100),100);
    }
};
int main(int argc,char** argv) {
    try {
        const ConfigFareTable configured{argc>1 ? argv[1] : "fares.txt"};
        if (configured.size()<3) throw std::invalid_argument("Demo requires at least three zones");
        const auto fare=configured.quote(2,8);
        metro::SmartCard card{10000};
        const bool allowed=card.deduct(fare);
        std::cout << "Configured zones: " << configured.size() << '\n';
        std::cout << "Outer peak fare Rs " << metro::money(fare) << '\n';
        std::cout << "Allowed=" << std::boolalpha << allowed << " balance Rs " <<
    metro::money(card.balance()) << '\n';
    } catch(const std::exception& error) {
        std::cout << "Configuration rejected: " << error.what() << '\n';
        return 2;
    }
    return 0;
}
```

Task12/main.cpp

```
#include "../include/metro_extended.hpp"
#include "../include/checks.hpp"
#include <iostream>
#include <string>
std::string describe(const extended::Quote& q) {
    return metro::money(q.fare)+",pass="+(q.passApplied?"1":"0")+",weekend="+(q.weekendApplied?"1":"0")+",pe
    ak="+(q.peakApplied?"1":"0");
}
int main() {
    using extended::Zone; using metro::Category;
    Checks check;
    const extended::FareRules rules;
    const extended::MonthlyPass none;
    const extended::MonthlyPass october{2026,10,Zone::Inner,Zone::Middle};
    const extended::MonthlyPass nonAdjacent{2026,10,Zone::Inner,Zone::Outer};
    struct Case { const char* name; extended::Journey journey; const extended::MonthlyPass* pass; long long
    fare; bool covered,weekend,peak; };
    const Case cases[]={
        {"Airport weekday
    offpeak",{Zone::Outer,Zone::Airport,{2026,10,20},10,0,Category::Regular},&none,6000,false,false,false},
        {"Airport weekday
    peak",{Zone::Outer,Zone::Airport,{2026,10,20},8,0,Category::Regular},&none,7200,false,false,true},
        {"Airport Saturday peak
    replaced",{Zone::Outer,Zone::Airport,{2026,10,24},8,0,Category::Regular},&none,5400,false,true,false},
        {"Airport Sunday
    offpeak",{Zone::Outer,Zone::Airport,{2026,10,25},10,0,Category::Regular},&none,5400,false,true,false},
        {"Student Airport
    weekend",{Zone::Outer,Zone::Airport,{2026,10,24},8,0,Category::Student},&none,4860,false,true,false},
        {"Senior Airport
    peak",{Zone::Outer,Zone::Airport,{2026,10,20},17,0,Category::Senior},&none,5760,false,false,true},
        {"Pass covered
    weekday",{Zone::Inner,Zone::Middle,{2026,10,20},8,0,Category::Regular},&october,0,true,false,false},
        {"Pass covered
    weekend",{Zone::Middle,Zone::Inner,{2026,10,24},8,0,Category::Student},&october,0,true,false,false},
        {"Pass last
    day",{Zone::Inner,Zone::Middle,{2026,10,31},23,59,Category::Regular},&october,0,true,false,false},
        {"Pass first
    day",{Zone::Inner,Zone::Middle,{2026,10,1},0,0,Category::Regular},&october,0,true,false,false},
        {"Pass next month
    expires",{Zone::Inner,Zone::Middle,{2026,11,1},8,0,Category::Regular},&october,1800,false,true,false},
        {"Pass wrong
    year",{Zone::Inner,Zone::Middle,{2027,10,20},8,0,Category::Regular},&october,2400,false,false,true},
        {"Pass uncovered third
    zone",{Zone::Middle,Zone::Outer,{2026,10,20},8,0,Category::Regular},&october,3600,false,false,true},
        {"Nonadjacent pass crosses
    Middle",{Zone::Inner,Zone::Outer,{2026,10,20},10,0,Category::Regular},&nonAdjacent,3000,false,false,fals
    e},
        {"Same selected zone
    covered",{Zone::Outer,Zone::Outer,{2026,10,20},10,0,Category::Regular},&nonAdjacent,0,true,false,false},
        {"Baseline Inner
    retained",{Zone::Inner,Zone::Inner,{2026,10,20},10,0,Category::Regular},&none,1000,false,false,false},
        {"Baseline Middle
    retained",{Zone::Inner,Zone::Middle,{2026,10,20},10,0,Category::Regular},&none,2000,false,false,false},
        {"Baseline Outer
    retained",{Zone::Middle,Zone::Outer,{2026,10,20},20,0,Category::Regular},&none,3000,false,false,false}
    };
    for (const auto& c:cases) {
        const auto actual=rules.quote(c.journey,*c.pass);
        const extended::Quote expected{c.fare,c.covered,c.weekend,c.peak};
        check.equal(c.name,describe(expected),describe(actual));
    }
    const extended::Journey covered{Zone::Inner,Zone::Middle,{2026,10,20},8,0,Category::Regular};
    const auto zero=rules.quote(covered,october).fare;
    metro::SmartCard reserve{5000},low{4999},airportCard{12200},insufficient{12199};
    check.equal("Pass at reserve accepted",true,reserve.deduct(zero));
    check.equal("Pass reserve unchanged",5000LL,reserve.balance());
    check.equal("Pass below reserve denied",false,low.deduct(zero));
    check.equal("Airport peak exact reserve",true,airportCard.deduct(7200));
    check.equal("Airport balance exact reserve",5000LL,airportCard.balance());
    check.equal("Airport one paisa short denied",false,insufficient.deduct(7200));
    check.equal("Airport denied balance unchanged",12199LL,insufficient.balance());
    bool unlimited=true;
    for (int i=0;i<1000;++i) if (!reserve.deduct(rules.quote(covered,october).fare)) unlimited=false;
    check.equal("1000 pass trips have no count limit",true,unlimited && reserve.balance()==5000);
    check.equal("2026-10-24 is weekend",true,extended::isWeekend({2026,10,24}));
```

```
    check.equal("2026-10-20 is weekday",false,extended::isWeekend({2026,10,20}));
    check.equal("Leap day 2024 valid",true,extended::validDate({2024,2,29}));
    check.equal("Nonleap day 2026 invalid",false,extended::validDate({2026,2,29}));
    check.equal("Century 1900 not leap",false,extended::validDate({1900,2,29}));
    check.equal("Century 2000 leap",true,extended::validDate({2000,2,29}));
    bool badDate=false,badZone=false,badTime=false,badPass=false,duplicatePass=false;
    auto invalid=covered; invalid.date={2026,2,29};
    try { (void)rules.quote(invalid,october); } catch(const std::invalid_argument&) { badDate=true; }
    invalid=covered; invalid.to=static_cast<Zone>(99);
    try { (void)rules.quote(invalid,october); } catch(const std::invalid_argument&) { badZone=true; }
    invalid=covered; invalid.hour=24;
    try { (void)rules.quote(invalid,october); } catch(const std::invalid_argument&) { badTime=true; }
    try { extended::MonthlyPass p{2026,13,Zone::Inner,Zone::Middle}; } catch(const std::invalid_argument&) {
     badPass=true; }
    try { extended::MonthlyPass p{2026,10,Zone::Inner,Zone::Inner}; } catch(const std::invalid_argument&) {
    duplicatePass=true; }
    check.equal("Reject invalid date even with pass",true,badDate);
    check.equal("Reject unknown fourth-zone enum",true,badZone);
    check.equal("Reject invalid time even with pass",true,badTime);
    check.equal("Reject invalid pass month",true,badPass);
    check.equal("Reject duplicate pass zones",true,duplicatePass);
    return check.finish();
}
```

Sources, provenance and viva preparation

- Assignment requirements: the user-supplied **ASSIGNMENT 1 -CO1-.pdf**, 13 pages. The original prediction programs and Task6 skeleton are assignment-provided material; working TODO implementations and other examples were prepared for this submission with Codex assistance.

- C++ working draft, usual arithmetic conversions: https://eel.is/c++draft/expr.arith.conv

- C++ working draft, integer division and remainder: https://eel.is/c++draft/expr.mul

- C++ working draft, list initialisation and narrowing: https://eel.is/c++draft/dcl.init.list

- GCC documentation, warning options: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html

The working draft is a current reference; the executed programs use C++14 on TDM64 GCC 4.9.2. The report distinguishes C++14 from later assignment sequencing rules rather than attributing later rules to the older compiler.

Actual results are recorded per task with compile diagnostics, input fixtures, captured stdout, execution timestamps, source hashes and exit statuses. `output.png` is a browser screenshot of `evidence.html`, displaying those real captured execution transcripts; it is not a native terminal screenshot. Build output is presented in reproducible repository-relative form. The working session used the same flags with absolute source/output paths and the compiler at `C:\Program Files (x86)\Dev-Cpp\MinGW64\bin\g++.exe`.

Commit timestamps reflect actual work in this session; the history is not backdated to imply a week of activity. Task4/Task5 prediction files and Task6 pseudocode were committed before the corresponding execution/implementation. The student must review and understand the assisted work and perform any independent verification required by the faculty.

The assignment requested a repository named DSA01-192471019-Assignment1. The student explicitly selected the existing course repository instead: https://github.com/hosheapaul2006/DSA0110_SLOT-B-192471019- The local deliverable folder uses the requested assignment name; the remote repository keeps its existing name. The exact separate SIMATS pseudocode style guide was unavailable; conventional structured notation is used, as approved.

Read the source and reproduce the tests before using these prompts. This guide does not assert that a viva or the student's own independent execution occurred.

1. **Why paise?** Explain exact integer representation, then show Task3's fractional-double accumulation and Task10's network totals. Integer money still has a range limit; demonstrate the checked int overflow boundary.

2. **Why not always double?** Rs36 is exact in binary at these counts but Rs28.80 is not. Formatting to two decimals does not fix the stored value.

3. **What happens at exactly 10 AM?** Evaluate the half-open interval and run Task6/boundaries.cpp. Then modify a test to 09:59 and explain the changed fare.

4. **Why is balance private?** Show Task7's public negative assignment and the unchanged balance on a denied private-card deduction.

5. **Why does -1 compare greater than unsigned 1?** Explain promotions/usual arithmetic conversions and the guarded explicit correction in Task4.

6. **Why does 7/2 assigned to double give 3?** Division happens before conversion. Cast an operand before division and explain the 3.5 result.

7. **Can braces catch every conversion mistake?** They reject implicit narrowing; an explicit cast can suppress that protection and still be wrong. Explain the deliberately rejected source and range validation before a cast.

8. **Does precedence specify evaluation order?** No. Use Task5's distinct x/y example and Task7's resolved printing-order problem to explain the difference.

9. **How do you recover after letters?** Show fail(), clear(), ignore(), strict suffix validation, and EOF termination in Task8. Compare getline/stoi.

10. **Why not subtract first and compare?** A huge or negative external fare could overflow or create credit. Explain the subtraction guard in canTravel().

11. **Modify a fare.** Edit one table entry, independently calculate new expected values, rebuild and run tests. Explain why changing tests to reuse quote() would remove the independent oracle.

12. **Why does weekend Airport peak cost Rs54?** The weekend rule replaces peak: 6000*90/100, not 6000*120/100*90/100. Student then costs Rs48.60 under the disclosed concession assumption.

13. **What does a monthly pass cover?** Explain calendar month/year, two distinct selected zones and all traversed zones. An Inner/Outer pass cannot cover a route through unselected Middle. Run expiry and nonadjacent-route cases.

14. **Why retain SmartCard in Task12?** Pricing changes, but zero/nonzero checked deductions still satisfy the same reserve invariant. Explain the pass-at- Rs50 and pass-below-Rs50 test results.

15. **What would production need?** Persistent accounts, atomic updates across gates, authenticated cards and versioned fare deployments. Local arithmetic measurements alone do not prove distributed throughput or security.

Suggested practice: run `python scripts/verify.py --task 9`, then change one boundary expectation deliberately, observe the failure, restore it and explain the diagnosis. Keep any genuine subsequent work as new commits; do not rewrite the recorded pre-execution predictions or invent historical work dates.